

Package Hallucinations as Phantoms in Open-source Software Supply Chains: An Empirical Security Analysis

ZHE HUANG, Beijing Institute of Technology, China and Zhongguancun Laboratory, China

CHAOGE LIU*, Zhongguancun Laboratory, China

ZHI WANG, Zhongguancun Laboratory, China

XIANG CUI, Zhongguancun Laboratory, China

XIAO DENG, Zhongguancun Laboratory, China

MENGYANG XU, Harbin Institute of Technology, China and Zhongguancun Laboratory, China

In code generation tasks, large language models (LLMs) frequently recommend nonexistent software packages, referred to as package hallucinations. This may pose a security risk, as attackers could register these fake packages and exploit them in supply chain attacks. To investigate the real-world feasibility, this paper conducts a systematic empirical study of this potential threat. We extracted 107 hallucinated packages from eight prominent LLMs and uploaded them to the npm registry over six months. The results demonstrate a tangible risk: 35.2% of package installations are attributed to developers, with most initial interactions occurring within a two-week window post-registration. A name similarity analysis further reveals that 25.2% of hallucinated names fall within an absolute edit distance of ≤ 1 from existing npm packages, posing a direct typosquatting risk. In our evaluated setting, tool-augmented generation, specifically using web search, fails to mitigate this threat. Instead, it significantly amplifies hallucination persistence from 11.2% to 17.3% through context contamination. In the npm registry, the entire attack campaign costs only an estimated \$496, making it far more accessible than traditional supply chain attacks yet capable of enabling poisoning effects.

To mitigate this emergent threat, we design an autonomous defense agent: NymGuard. It provides a proactive defense by automating the real-time detection of hallucinated packages, their preemptive registration to block malicious use and the maintenance of a public warning registry. The full implementation has been open-sourced and is available on GitHub (<https://github.com/xiaoye798/Package-Hallucination-Research>).

CCS Concepts: • **Security and privacy** → Software security engineering; **Spoofing attacks**; • **General and reference** → **Empirical studies**.

Additional Key Words and Phrases: Hallucination, Software Supply Chain Security, Large Language Models

1 Introduction

In recent years, large language models (LLMs) have significantly reshaped the landscape of software development by producing syntactically valid code, automating workflows and assisting with complex algorithms [3, 12]. The

*Corresponding author

Authors' Contact Information: Zhe Huang, Beijing Institute of Technology, Beijing, China and Zhongguancun Laboratory, Beijing, China, 3120235245@bit.edu.cn; Chaoge Liu, Zhongguancun Laboratory, Beijing, China, liucg@zgclab.edu.cn; Zhi Wang, Zhongguancun Laboratory, Beijing, China, wangzhi@zgclab.edu.cn; Xiang Cui, Zhongguancun Laboratory, Beijing, China, cuix@zgclab.edu.cn; Xiao Deng, Zhongguancun Laboratory, Beijing, China, dengx@zgclab.edu.cn; Mengyang Xu, Harbin Institute of Technology, Harbin, China and Zhongguancun Laboratory, Beijing, China, 15810827912@163.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/7-ART

<https://doi.org/10.1145/3830237>

systemic integration of LLM-based assistants into modern development ecosystems has significantly boosted productivity, with developers using tools like GitHub Copilot [20, 38] completing tasks 55% faster than those without, alongside reducing deployment cycles and accelerating innovation across domains such as full-stack development. Despite these advantages, LLMs exhibit a significant limitation in the form of hallucinations [18, 19, 51, 55], which raise severe security concerns by potentially introducing risks into code generation pipelines [15, 41, 52].

This paper focuses on a largely underexplored type of hallucination called package hallucinations. During code generation tasks, LLMs sometimes recommend nonexistent software packages that are not registered in package repositories. This issue intersects directly with the security of the Open Source Software (OSS) supply chain [4, 25, 47] and its broader development lifecycle [1, 48]. Modern ecosystems rely heavily on repositories like npm [39, 54] and their associated CLI tools [36]. The security of this infrastructure rests on a fragile trust in package names as identifiers. While established name-based attacks like typosquatting [35, 40, 56] and dependency confusion [17, 23] already exploit this vulnerability, package hallucinations establish a novel attack vector weaponizing this exact same risk [5, 30, 50].

Malicious actors can strategically preregister these hallucinated package names, thereby transforming helpful outputs into potent malware distribution channels. LLMs act as a powerful catalyst and a uniquely persuasive delivery mechanism, turning a passive trap into an active and authoritative recommendation. The diagram of this exploitation is shown in Figure 1.

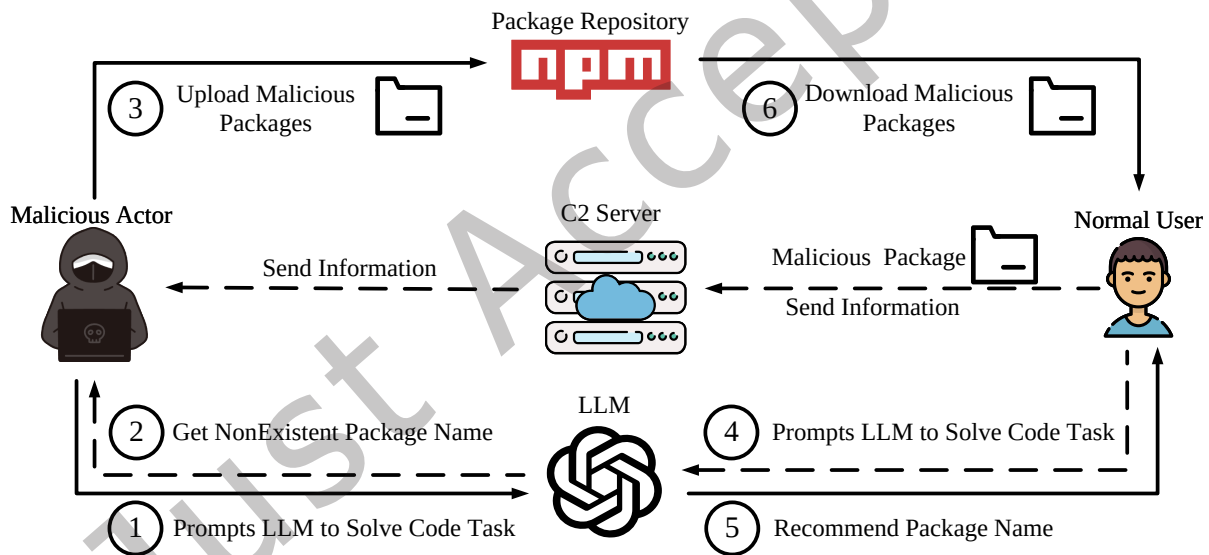


Fig. 1. Exploiting Package Hallucinations: An Example of Sensitive Information Collection

As discussed in Section 2, existing research has primarily concentrated on measuring package hallucinations and developing mitigation. Their work fails to address an important question: **Do hallucinated packages represent a substantive threat to software supply chains, or are they merely a theoretically impractical concern?** Addressing this gap is essential to understanding the practical implications of hallucinated packages and designing effective countermeasures. In this work, we seek to bridge this gap through a two-pronged approach that combines empirical investigation with innovative mitigation strategies. The key contributions of this paper are:

- We conduct an in-the-wild evaluation to assess the security risks posed by package hallucinations, simulating attacker strategies by uploading hallucinated packages generated by prominent LLMs.
- We provide a multi-dimensional threat characterization of 107 hallucinated packages. Our analysis reveals that 35.2% of installations are human-driven and that 25.2% of hallucinated names fall within an edit distance of ≤ 1 from existing packages, posing typosquatting risks. In our evaluated setting, tool-augmented generation, specifically using web search, counterintuitively amplifies hallucination persistence rather than mitigating it.
- We develop an autonomous agent: NymGuard. It is designed to detect hallucinated packages in real-time, dynamically register placeholder packages to preempt malicious exploitation and maintain a warning list.

Ethics Considerations. The design and execution of this research strictly adhere to established academic ethical guidelines. Necessary measures have been implemented to ensure the study adheres to principles of safety.

- (1) **Principle of Placeholder Packages:** All experimental packages registered and uploaded to npm in this study are placeholder packages. These packages are meticulously designed to ensure they anonymize user information and have no functions that could compromise privacy or lead to unintended behavior.
- (2) **Minimizing Disruption to the Package Management Ecosystem:** Packages registered during the experiment are derived from hallucinated names generated by LLMs, rather than preempting existing legitimate packages. Upon completion of experimentation, a report on hallucinated packages will be submitted to npm.
- (3) **Societal Value of the Research:** The fundamental aim of this research is to reveal potential security risks from package hallucinations in LLMs and to explore effective defense mechanisms, thereby contributing to the overall security of the software supply chain rather than providing attackers with a new attack vector. The findings are anticipated to provide valuable insights for developers, security researchers, and LLM providers.

2 Related Work

A significant gap exists in the research literature regarding the analysis of package hallucinations as a practical and end-to-end attack vector. Current research is predominantly confined to two areas: measuring package hallucinations and developing mitigation strategies.

2.1 Measuring Package Hallucinations

There are several studies related to package hallucinations. The possibility of code-generating models recommending malicious or typosquatted packages was first proposed in 2020, coinciding with the release of platforms like GPT-3 and Codex [6]. Notably, package hallucinations were not proposed in this study. This was because the use of generative AI was not mainstream at that time. In 2024, Spracklen et al. [42] officially revealed package hallucinations to the academic field by systematically measuring their rate across 16 models, reporting an average of 19.7% for Python and JavaScript. They also investigated mitigation strategies summarized as pre-generation techniques, such as Supervised Fine-tuning (SFT) and Retrieval-Augmented Generation (RAG). While pre-generation techniques aim to prevent or reduce the possibility of package hallucinations occurring, they also have disadvantages. For example, SFT requires significant resources, RAG depends on external knowledge sources that must be meticulously maintained and self-refinement lacks sufficient reliability. In 2025, Krishna et al. [22] analyzed 11 models and found hallucination rates ranging from 0.22% to 46.15% across Python, JavaScript and Rust, noting an inverse correlation between model size and hallucination frequency.

In summary, the above research collectively illustrates the prevalence and characteristics of package hallucinations across various LLMs and programming languages. Their primary focus is on systematic measurement, quantifying key aspects such as hallucination frequency and exploring influencing factors like model scale and

target language. They provide valuable quantitative data and establish the phenomenon within LLM-driven code generation.

However, a crucial gap is common to these existing works. Their predominant focus is on phenomenon measurement and theoretical risk. **While effectively demonstrating that LLMs hallucinate package names and logically explaining why this could pose a security threat, they lack empirical validation of these risks in real-world environments.** If registered by malicious actors, the extent to which hallucinated packages would actually be installed thereby posing a concrete security threat remains largely unexplored experimentally. This gap between the theoretical potential and demonstrated real-world impact underscores the necessity for empirical investigation.

2.2 Existing Mitigation Strategies

Recent researches [42, 46] have primarily focused on either mitigating hallucination before code generation or detecting it after. While these efforts provide valuable insights, they have limitations.

Post-generation Techniques: These methods [46] typically involve analyzing output generated by LLMs. A common approach involves comparing the recommended package names against a list of valid packages from official registries. This defense strategy is fundamentally flawed and ineffective against attackers. Because an attacker can swiftly register malicious packages under those exact names on public registries. Once registered, these malicious packages then appear on the list of valid packages, making the comparison mechanism useless. LLMs would then recommend an existing yet malicious package.

Pre-generation Techniques: These methods, such as SFT [16, 28, 45] and RAG [11, 27, 53], aim to prevent LLMs from generating hallucinated outputs or to correct hallucinations before they are presented to the user. While SFT can reduce hallucinations by training models on carefully curated and labeled datasets, it is exceptionally resource-intensive. It requires vast amounts of meticulously labeled data, significant computational resources for training and continuous retraining to adapt to new packages or evolving coding practices. This makes it impractical for agile and real-time defense. Furthermore, RAG integrates external knowledge bases to constrain the generation of models to existing and legitimate packages. However, RAG inherently depends on these external knowledge sources being meticulously maintained, comprehensive and up-to-date. Any delay in updating the knowledge base or gaps in its coverage can still lead to hallucinations or the recommendation of outdated/vulnerable packages. Furthermore, approaches using API documentation to reduce code LLM hallucinations have been explored, involving checks on model confidence and content verification [21]. Similarly, iterative retrieval of relevant APIs aims to improve prompts and decrease hallucinations, but still faces issues with updating knowledge sources. More recent work has moved closer to the generation process itself. PackMonitor [29], for example, performs decoding-time monitoring by detecting package mentions as they are generated and constraining the output before hallucinated package recommendations reach developers. This approach is well motivated because it reduces exposure at the source. However, because PackMonitor intervenes synchronously during decoding via token-level validity checking and logits masking, it is not readily applicable to fully black-box settings and is instead mainly suited to environments where the inference stack can be instrumented. NymGuard addresses a different part of the risk surface by focusing on downstream registry-side protection when model-side control is unavailable or incomplete, as with black-box LLMs.

In summary, existing solutions either demand significant resources or fail to provide a truly robust and proactive defense against malicious package squatting.

3 Methodology

This section details the methodology employed to systematically investigate package hallucinations. We first present our overall research approach, which combines large-scale empirical analysis with security modeling.

Following this, we define a formal threat model to deconstruct the attack pathway and contextualize the action of adversaries. Finally, guided by this framework, we formulate the specific research questions (RQs) that drive the subsequent experiments and analysis presented in this paper.

3.1 Research Approach

Figure 2 illustrates our approach towards verifying package hallucinations and designing a large-scale hallucinated package defense agent. This study involves three crucial stages as below:

- (1) **Simulate Attack**: Specifically, it aims to simulate the behavior of attackers by proactively registering packages under names hallucinated by various LLMs within the package management ecosystems such as npm;
- (2) **Check Analysis**: The experimental results, detailed in Section 5, confirm the threat posed by the hallucinated package and provide a summary of its characteristics and threat model;
- (3) **Automatic Mitigation**: The characteristics of hallucinated packages make conventional mitigation methods difficult to apply. Therefore, this paper proposes NymGuard comprising hallucination detection, package squatting and real-time warning modules.

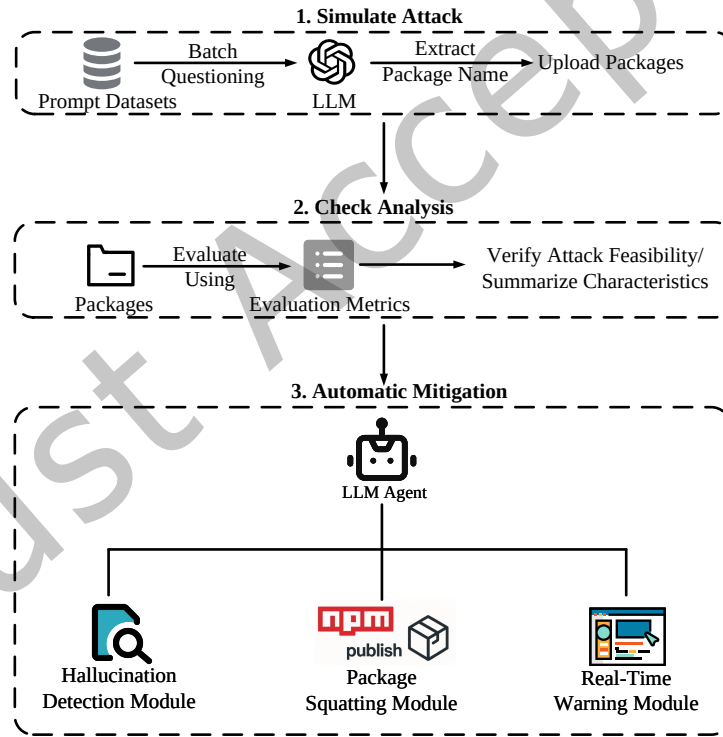


Fig. 2. Overview of This Study

3.2 A Three-Phase Attack Kill Chain

To deconstruct this potential threat, we propose a three-phase kill chain framework comprising the core attack mechanism and a subsequent amplification strategy. The three phases form the core attack path, outlining the interdependent steps from package hallucinations to payload delivery that are essential for the attack to succeed.

- **Phase 1: Hallucination & Exposure.** The initial phase is triggered when an LLM fabricates a plausible but nonexistent package in response to a code generation prompt. We established the prevalence of this phenomenon through a large-scale experiment involving eight commercial LLMs and 10,893 queries. This process yielded 107 hallucinated packages, demonstrating that the initial condition for this threat is a frequent occurrence.
- **Phase 2: Attacker Malicious Registration.** In this phase, an attacker identifies hallucinated packages and proceeds to register malicious packages under these names in the corresponding registry. To simulate this malicious behavior, our study involved registering and uploading 107 placeholder packages using the names previously identified, as discussed in Section 4.3.
- **Phase 3: Malicious Delivery & Sustained Propagation.** This phase is triggered when real users follow the recommendation from LLMs and attempt to install the package. Because the attacker has already registered these names, the installation succeeds, delivering the malicious payload from the attacker. In this process, LLM serves as an unwitting vector for malicious code distribution. Our findings confirm that real users are susceptible to this. For five of the 107 packages we uploaded, over a quarter of the installation attempts came from user IP addresses, as discussed in Section 5.1. This result clearly shows that developers do attempt to install these hallucinated packages.

3.3 Research Questions

To transition from theoretical possibility to experimental assessment, this paper investigates the question: **Do hallucinated packages represent a substantive threat to OSS supply chains or are they merely a theoretically impractical concern?** To answer this, we deconstruct the threat into a three-phase attack kill chain, allowing for a systematic evaluation of its entire lifecycle. Each phase, from the initial generation of hallucinated names to potential weaponization via social engineering, presents a critical checkpoint for the attack feasibility. Accordingly, we structure our investigation around the following RQs, with each question corresponding to a specific phase of this kill chain.

RQ1: Do packages, registered under LLM-hallucinated names, attract human interactions when published to package repositories (Phase 1)? This question explores the fundamental viability of this attack vector: whether the hallucinated names generated by LLMs, once registered, actually trigger installation attempts.

RQ2: Once an adversary registers packages under hallucinated names (Phase 2), what is the typical latency of it being delivered to unsuspecting developers through installation attempts (Phase 3)? This analysis assesses the immediacy of the threat posed by hallucinated packages. Such an assessment is crucial for understanding how quickly attackers can gain a foothold and the response window available for effective mitigation.

RQ3: What observable characteristics do the two super-spreader packages share, and to what extent might these characteristics be associated with their disproportionately higher propagation compared to the majority of packages that see minimal adoption? This investigation serves as an initial step toward understanding what package attributes may correlate with elevated propagation risk.

RQ4: To what extent do LLM-hallucinated package names mimic existing naming conventions in the ecosystem, and how does this overlap facilitate typosquatting-like security risks? This question quantifies the structural and character-level similarity between fabricated names and legitimate packages to determine if hallucinations inadvertently create typosquatting risks.

RQ5: To what extent can tool-augmented generation, such as live web search or registry API verification, mitigate or eliminate package hallucinations during code generation? It investigates whether hallucinations are simply a result of information deficits or if they are deeply rooted in model architecture, potentially persisting or even increasing despite tool-based interventions.

RQ6: Considering the entire three-phase kill chain, what are the defining characteristics and operational costs that determine the overall viability of the LLM-hallucinated package threat in a real-world software supply chain? Our aim here is to establish a comprehensive understanding of the unique threat model posed by LLM-hallucinated packages, providing crucial insights for the development of targeted defense strategies.

4 Experiment Design

To address the aforementioned research gap in Section 2.1, this section details the experimental methodology used to quantify the propagation risk of LLM-hallucinated packages. Our experimentation pipeline consists of three phases: (1) language/dataset selection; (2) package name extraction; (3) simulated attack.

4.1 Language and Dataset Selection

4.1.1 Language Selection. In this study, we choose JavaScript as the subject of analysis because of its large ecosystem and the urgent security risks in npm. JavaScript relies on npm, the largest open-source package platform, which hosts millions of public packages and serves more than 17 million developers. Its centralized structure and wide adoption make npm a high-value target for software supply chain attacks.

A notable example occurred in September 2025, when attackers compromised 18 popular npm packages, including “debug” and “chalk” [10]. Together, these packages have more than 2 billion weekly downloads, widely used in both frontend (e.g., React) and backend (e.g., Node.js) projects.

This incident shows how broad the impact of JavaScript/npm supply chain attacks can be, and why studying dependency management and vulnerabilities in this ecosystem is both representative and crucial for improving real-world supply chain security.

4.1.2 Prompt Dataset. To obtain a realistic and large-scale set of developer queries, we adopted a specific subset of the prompt dataset from Spracklen et al. [42]. This subset consists of 10,893 JavaScript-related questions sourced from Stack Overflow [43], representing a diverse range of common developer tasks.

While Spracklen et al. [42] used this dataset to measure the prevalence of hallucinations, our objective is to analyze the propagation factors of these packages (RQ3), specifically whether the type of task influences adoption. To this end, we conducted a novel semantic classification of the entire 10,893-query dataset.

Using automated keyword matching, we categorized each query into functional domains. The detailed methodology, classification standards, and keywords are available in Appendix B.2. Our analysis revealed that **63.5%** (6,916) of queries related to **Data Extraction** (e.g., parsing files, scraping HTML), while only **24.8%** (2,704) related to **Statistical Analysis** (e.g., plotting, data modeling). This classification provides a data-driven baseline for understanding task popularity, which we use in Section 5.3.3 to analyze the “super-spreader” phenomenon.

4.2 Package Name Extraction and Verification

We then fed these 10,893 classified prompts to a diverse set of state-of-the-art models, including top-ranked entries from the Chatbot Arena [7], to ensure our findings are generalizable, as seen in Table 16. Using official APIs and default parameters, we prompted the models with a specific system message instructing them to format packages as “npm install xx”. To rigorously differentiate model-driven fabrication from confounding factors such as ecosystem churn, we established a comprehensive existence status taxonomy. All 107 candidate packages were subjected to a multi-stage verification pipeline. The pipeline follows a four-step protocol:

- (1) Querying the **npm public registry API** for current existence status;

- (2) Cross-referencing the **Libraries.io** historical index [34], which aggregates and preserves metadata from package registries, enabling the detection of packages that were previously published but have since been unpublished, deprecated or renamed;
- (3) Scanning the **PyPI API** and **GitHub** repository search for cross-ecosystem presence;
- (4) Checking **10 common npm scopes** (e.g., @types/, @babel/, @vue/) for namespace variants.

Based on these audit outcomes, we establish a taxonomy consisting of four mutually exclusive types: **Pure Fabrication**, **Historical Artifact**, **Cross-Ecosystem Leakage**, and **Namespace Mismatch**. Table 1 details the mapping between verification criteria and resulting classifications. Notably, zero packages fell into **Historical Artifact** or **Namespace Mismatch**, indicating that ecosystem evolution contributes negligibly to observed non-existence. Critically, all 107 packages are confirmed non-existent in the target npm ecosystem. Of these, 88 packages are Pure Fabrications with no counterpart in any ecosystem, while the remaining 19 represent Cross-Ecosystem Leakage, where the model erroneously recommended Python packages for JavaScript.

Table 1. Existence Status Taxonomy and Classification of 107 Packages

Classification	Verification Criteria	Count
Pure Fabrication	Negative in current, historical, and cross-registry audits	88
Historical Artifact	Currently absent (404), but previously published, re-named or transferred per historical archives	0
Cross-Ecosystem Leakage	Exists in PyPI/GitHub; absent from npm registry	19
Namespace Mismatch	Only exists as a scoped variant (e.g., @scope/name)	0

4.3 Simulate Attack

To empirically assess the real-world threat, we simulated an attack by registering the identified hallucinated packages on npm. To safely track genuine installations, these packages were configured to execute a standard *postinstall.js* script, which transmitted anonymized telemetry data to our callback server upon installation. Furthermore, to mimic the diverse and unpredictable tactics of real-world adversaries, we implemented a strict “one-package, one-account” operational strategy and deployed the packages across multiple randomized batches rather than a single large-scale submission. This approach was designed to mimic the diverse and unpredictable tactics attackers might employ in real-world scenarios.

5 Evaluation Results

In this section, the results of our experimental analysis will be presented to answer RQ1-RQ6. Over six months, 107 software packages were released, designated with serial numbers from 1 to 107. For details regarding the file structure of each package, please refer to Section 4.3. As shown in Table 2, the collected data are analyzed along several dimensions using specific metrics to understand the propagation and potential risks. This analysis aims to quantify the propagation characteristics, profile the entities interacting with these packages and ultimately assess the tangible security risks posed by this phenomenon.

5.1 IP Address Classification (RQ1)

Based on the metrics for profiling downloaders and inferring usage scenarios, a set of classification rules have been synthesized to systematically evaluate and categorize downloader activities and their operational contexts.

Our classification rules and illustrative examples, which are detailed in Appendix A, rely on a comprehensive analysis of static attributes, system characteristics, and behavioral patterns. Static attributes primarily concern

Table 2. Metrics for Analyzing Hallucinated Packages

Dimension	Sub-Metric	Description
Metrics for Analyzing Propagation Speed and Scope	Number of Unique User IP Addresses	Counts the distinct source IP addresses from which installation callback requests were received
	Time-to-First-Download (TTFD)	Measures the time difference between the publication of a package and the first installation callback from a user IP address
	Download Rate and Temporal Trends	Involves analyzing the frequency of unique installations over time to identify patterns such as initial spikes and sustained activity
Metrics for Profiling Downloaders	Geographical Distribution	IP addresses will be geolocated to determine the country level
	Operating System Distribution	Determines the operating systems where packages were installed
	System Environment Categorization	Involves inferring the type of system environment (e.g., server, container, cloud) by analyzing metadata
Metrics for Inferring Usage Scenarios	IP Attribution Analysis	Publicly available data is used to classify the network origin of each IP address (e.g., residential or cloud service provider)
	Download Timing	Installation timestamps will be analyzed to distinguish between developer activity and automated processes

intrinsic information about the IP address, such as its reputation score, affiliation with cloud service providers, and network context (e.g., corporate, data center, or residential ISP). System environment characteristics are inferred from a collection of indicators to distinguish between automated and user-centric environments. Key indicators include the operating system type (server-grade vs. desktop-grade), hardware specifications like CPU architecture, and the presence or absence of a graphical user interface. Finally, behavioral and temporal dynamics focus on activity patterns, such as the frequency and duration of connections and session interactivity (scripted vs. human-driven), to determine if the activity is user-initiated.

It is worth noting that our classification method has limitations due to ambiguity in IP origins. For example, developers using cloud IDEs (such as GitHub Codespaces or AWS Cloud9) access remote environments through browsers or clients, while SSH workflows involve logging into servers for development. In these cases, IP addresses may come from cloud providers like AWS or Azure, which resemble those from automated scripts or CI/CD pipelines. This overlap can lead to misclassification, potentially underestimating the developer proportion.

5.1.1 Expert Classification and Agreement. The entire classification process was performed manually by two security researchers, each with over five years of professional experience in network traffic analysis. Using the rules in Appendix A, each expert independently classified all 343 IP addresses into one of five categories: *Real User*, *Automated Script*, *Weak Script Signal*, *Undetermined*, or *Weak User Signal*. To quantify the consistency of expert judgments, we calculated Cohen’s κ coefficient [8], which was 0.88, indicating a strong agreement level

(>0.80). The raw percent agreement between the two experts was 94%. All disagreements were resolved through discussion. These results demonstrate a high inter-rater reliability in the manual classification process, providing a reliable annotated basis for subsequent data analysis. The classification result is shown in Figure 3. Based on the IP classification stats, a key observation emerge:

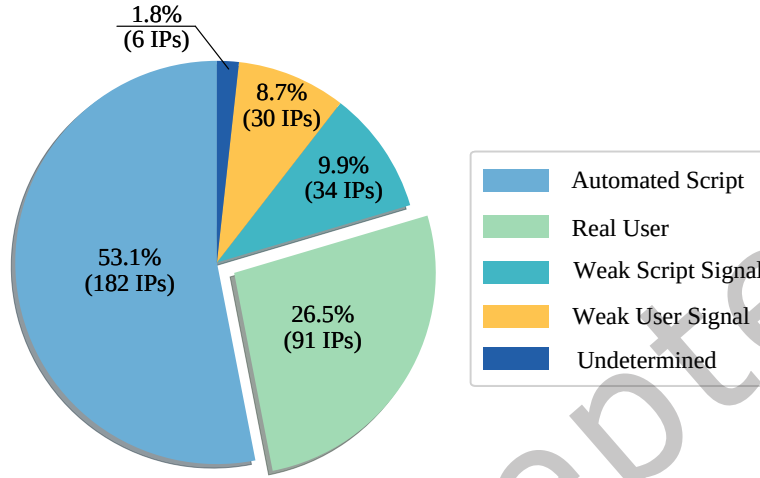


Fig. 3. IP Address Attribution Results

• **Finding 1: A significant portion of package interactions (35.2%) are driven by human developers, validating the threat as an observed, high-impact attack surface.** This high rate of engagement moves the concern from a theoretical risk to a tangible phenomenon. The fact that more than a quarter of interactions are user-driven directly confirms that real developers are encountering and installing these packages, which is particularly consequential as even a single manual installation can propagate through dependency manifests into CI/CD pipelines and production releases [13, 14, 44].

RQ1 Summary: By attracting a few human-driven installations, packages registered with LLM-hallucinated names transform a theoretical attack vector into a validated real-world threat.

5.2 Survival and Time Latency Analysis (RQ2)

5.2.1 Hallucinated Packages Survival. The survival times of 107 hallucinated packages are summarized in Figure 4. It can be seen that many packages remained active for nearly two weeks. Such a prolonged duration creates a substantial window of opportunity for attackers. Package survival times and takedown patterns reveal critical insights into the security posture of npm:

• **Finding 1: npm security responses exhibit systematic and batch-like behavior rather than continuous, real-time monitoring.** The observation of a distinct cluster of 33 packages being removed after an identical 12-day lifespan indicates a coordinated takedown event. This phenomenon points towards a systematic security process, such as a periodic, batch-based cleanup or a delayed response to a detected common threat pattern, rather than random or immediate discovery.

• **Finding 2: The predictability of the security response creates a reliable and exploitable window for attackers.** This consistent 12-day survival window, while demonstrating an eventual cleanup capability,

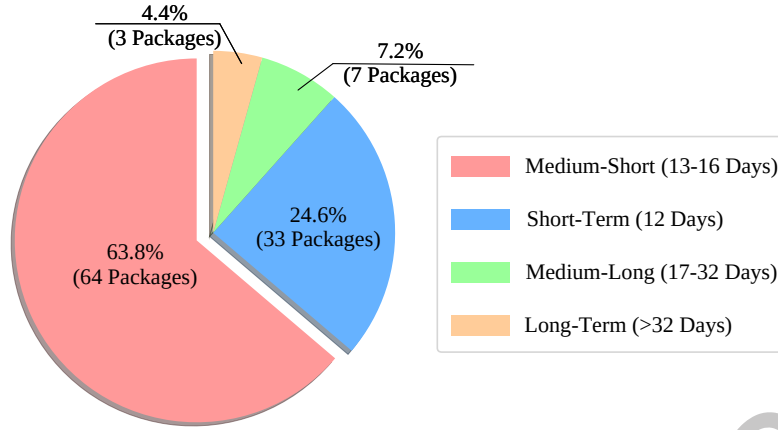


Fig. 4. Package Survival Time Distribution

establishes a predictable timeframe for adversaries. Sophisticated attackers can leverage this knowledge of the platform’s operational cycles to plan their activities, ensuring their malicious packages remain active for a known duration before a likely takedown, thereby maximizing their potential impact.

5.2.2 Time Latency Analysis. Based on the metrics for analyzing propagation speed and scope, a time latency analysis was conducted on the 91 user IP addresses that were previously classified (detailed in Section 5.1) as having interacted with the experimentally deployed, hallucinated packages. As shown in Figure 5, over 70% of all IP interactions occurred within the two weeks. This activity was most intense in the first week, which accounted for 40.22% of interactions, while the second week accounted for an additional 30.43%. Our investigation into interaction patterns with these packages reveals a risk profile characterized by two primary factors:

- **Finding 3: The risk profile follows a dual pattern, with an acute, high-velocity risk window in the first two weeks followed by a significant long-tail threat.** The initial 14-day period constitutes the critical window for the propagation and poses the highest risk of developer exposure. However, the threat is not confined to this period. A notable long-tail effect exists, with nearly a quarter of all interactions occurring after this initial peak. This demonstrates that packages surviving the initial takedown attempts establish a persistent and lingering threat vector.

- **Finding 4: The threat is not uniformly distributed: a few “super-spreader” packages create a highly concentrated risk.** The vast majority of developer interactions were focused on just two packages (2 and 53). This finding indicates that the overall risk is disproportionately driven by a few successful packages that achieve significantly propagation. This concentration means that while many packages may pose a low-level threat, the most acute risk stems from a few high-impact packages.

RQ2 Summary: The typical delivery occurs within an acute two-week window of high-velocity risk, but this transitions into a significant long-tail threat, a direct result of systematic latencies in the security response.

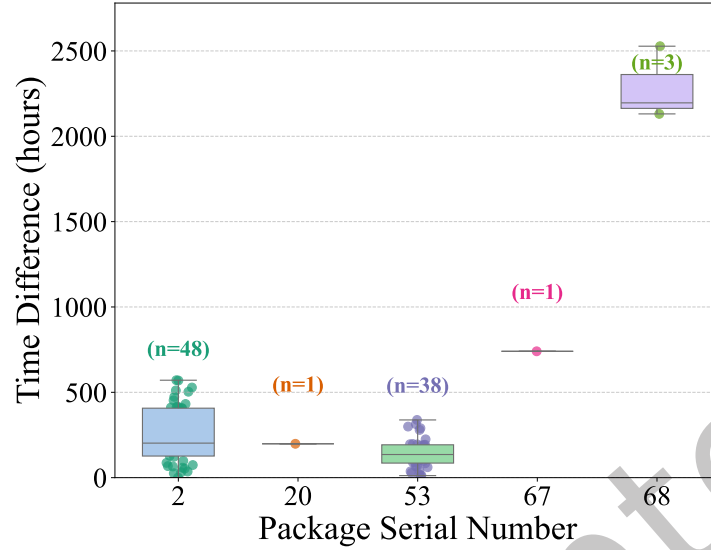


Fig. 5. User Time Latency Distribution (Each box plot shows the distribution of time differences for a specific package serial number. The central line indicates the median, the box represents the interquartile range (IQR), and the whiskers extend to $1.5 \times \text{IQR}$. Individual data points (IP addresses) are overlaid as colored dots to show the precise distribution. The value of n annotated above each plot represents the total number of unique IP addresses.)

5.3 Illustrative Analysis of the Two Super-Spreader Cases (RQ3)

As established in Section 5.2, only two hallucinated packages: *pdfplumber* and *beautifulsoup4* were observed to have notable propagation behavior. All other hallucinated packages did not reach significant adoption thresholds and thus did not qualify as super-spreaders. Given that only two cases are available, this subsection provides an illustrative comparison of their shared characteristics rather than a causal analysis. We emphasize that observations drawn from two cases are necessarily tentative and serve to inform future investigation, not to establish general conclusions.

5.3.1 Factor Definition. To investigate why these two packages achieved super-spreader status, we analyze their characteristics along three key dimensions: (1) Brand Effect, reflecting cross-ecosystem recognition and perceived credibility; (2) Task Popularity, representing the frequency with which their associated tasks occur in developer queries; (3) Repeated Hallucination Frequency, indicating the rate at which the same or highly similar non-existent package names are consistently generated across independent queries.

5.3.2 Package Classification. We classified the 107 identified hallucinated packages from Table 12 into four categories based on their naming features, detailed in Table 3. The classification process combined automated keyword matching and manual review, as described in Appendix B.1.

The distribution of these categories across 107 hallucinated packages is not uniform. As shown in Table 12, the Cross-Ecosystem Brand Type [2, 24] accounted for 49.53% (53/107) of the total, and the Hybrid Type accounted for 37.38% (40/107). Theoretically, these brand-related types are presumed to have a higher propagation potential, as they are more likely to be trusted by developers than the Placeholder or Generic New Name types. However, given that only two packages achieved “super-spreader”, it is clear that the brand effect alone is insufficient to explain propagation.

Table 3. Categories and Features of Hallucinated Packages

Category	Key Features	Potential Spread
Cross-Ecosystem (C)	Same name as a well-known project in another ecosystem, not in npm	High: strong brand recognition
Hybrid (H)	Contains partial brand terms	Medium: name carries trust signals
Generic New Name (G)	Mimics existing naming patterns	Low: lacks brand effect
Placeholder (P)	Obvious template or placeholder	Very low: no brand meaning

5.3.3 Illustrative Comparison of Shared Characteristics. While 53 of the 107 packages (49.53%) possessed a strong “Cross-Ecosystem Brand”, the vast majority failed to gain traction. With only two super-spreader cases available for comparison, any explanation is necessarily preliminary. We observe that *pdfplumber* and *beautifulsoup4* share the following three characteristics, which may have contributed to their disproportionate adoption:

- **Brand Trust as a Prerequisite:** Both packages are Type C, borrowing significant credibility from their namesakes in Python. Developers, particularly those with cross-language experience, are likely to perceive *npm install beautifulsoup4* not as a hallucination. This inherited trust (Brand Effect) effectively bypasses initial developer skepticism.
- **Task Domain Plausibility as the Trigger:** Crucially, both packages fulfill functions that are highly common and plausible within the target JavaScript/Node.js ecosystem. HTML/XML parsing and PDF data extraction are frequent tasks in web scraping, data processing, and backend development. Conversely, other Type C packages like *seaborn* (No. 44) or *dplyr* (No. 82), which, while popular in Python/R, map to statistical analysis tasks less commonly performed in a typical Node.js environment. This observation is supported by our dataset, where queries related to data extraction (e.g., “parse HTML”, “read PDF text”) are significantly more prevalent (representing 63.5% of task-oriented queries) than those related to statistical analysis (e.g., “plot data”, “statistical model”), which accounted for only 24.8%.
- **Cross-Model Exposure as a Potential Amplifier:** As shown in Table 4, these two packages exhibit distinct exposure patterns. *pdfplumber* is hallucinated by 5 different models, while *beautifulsoup4* is generated by only a single model, yet both still achieved super-spreader status. This divergence suggests that, if the observed characteristics do play a role, super-spreader status might emerge through more than one pathway — for instance, multi-model amplification as seen with *pdfplumber*, or strong intrinsic plausibility driven by brand trust and task-domain alignment as seen with *beautifulsoup4*. However, with only two cases, we cannot determine whether these represent general pathways or simply idiosyncratic features of the two observed packages.

Furthermore, the “Survival Days” metric in Table 4 suggests a potential correlation between exposure time and propagation patterns. *pdfplumber*, with a longer existence for 32 days compared to *beautifulsoup4* for 15 days, aligns with its observed “long-term sustained spread”, whereas the newer *beautifulsoup4* exhibited a “short-term rapid spread”. We note that exposure time alone does not appear sufficient for super-spreader status, as several other packages with comparable survival times attracted minimal adoption. Whether exposure time acts as a necessary enabling condition alongside the characteristics described above remains an open question.

Table 4. Key Metrics for *beautifulsoup4* and *pdfplumber* Packages

Metric	<i>beautifulsoup4</i>	<i>pdfplumber</i>
Model Count	1	5
Survival Days	15	32
Type	C (Cross-Ecosystem Brand)	C (Cross-Ecosystem Brand)
Functional Domain	General Programming (HTML/XML Parsing)	Data Processing & Analysis
Observed Propagation Pattern	Short-term rapid spread	Long-term sustained spread
Cross-Ecosystem Origin	Python HTML/XML parsing library	Python PDF processing library

RQ3 Summary: An illustrative comparison of the two super-spreader packages reveals that both share cross-ecosystem brand recognition and high task-domain relevance. However, with only two cases, no general conclusions about causal factors can be drawn.

5.4 Name Similarity and Typosquatting Risk Analysis (RQ4)

We performed a comparative name similarity analysis to evaluate the security implications of package hallucinations. By cross-referencing 107 hallucinated packages identifiers with 3.8 million legitimate packages in npm, we assessed the extent to which LLM-generated outputs mimic existing package names.

5.4.1 Evaluation Metrics. To systematically quantify this similarity, we employ three complementary metrics: Edit Distance, Token Overlap and Naming Pattern Classification.

Normalized Edit Distance: We compute the Levenshtein distance between each hallucinated name and its closest match in npm. Levenshtein distance is a measure of how many insertions, deletions, and substitutions are required for two strings to match[26]. Specifically, for each one, we search for the closest match among the roughly 3.8 million npm packages available as of February 2025. We first compare the hallucinated packages against npm packages with similar lengths (within a dynamic tolerance of $\max(5, \lfloor L/3 \rfloor)$ characters, where L is the length of the hallucinated packages). This filter efficiently speeds up the search across 3.8 million packages by excluding names with vastly different lengths. The formula provides a baseline window of 5 characters for short names, and scales proportionally ($\lfloor L/3 \rfloor$) for longer ones without missing potential matches. If fewer than 100 candidates remain after filtering, expand the search to the full registry. The Levenshtein distance is then divided by the maximum length of the two compared names to produce a normalized score between 0 and 1:

$$\text{Normalized Edit Distance} = \frac{\text{Levenshtein Distance}(h, n)}{\max(|h|, |n|)} \quad (1)$$

where h is the hallucinated package, n is the closest matching npm package. $|h|$ and $|n|$ denote their respective lengths. A score of 0 indicates identical strings, while a score approaching 1 indicates completely different strings.

Token Overlap: To capture semantic and structural similarities beyond character-level edits, we compute the Jaccard similarity coefficient between tokenized package names. This involves decomposing identifiers using standard delimiters (hyphens, underscores and dots) into distinct token sets. For example, `@babel/core` is tokenized as `{babel, core}` and `mongoose-to-json` as `{mongoose, to, json}`. The Jaccard similarity is then calculated as $J(A, B) = |A \cap B| / |A \cup B|$, where A and B are the token sets of the hallucinated and existing package names, respectively.

Naming Pattern Classification: We selected six structural naming patterns commonly used in npm based on npm documentation, including patterns such as `-plugin` (suffix), `node-` (prefix), `-js` (suffix) and `@scope/name`.

We categorize each hallucinated name by whether it follows these patterns and analyze their prevalence across different package categories (C, H, G, P as defined in Section 5.3.2).

5.4.2 Results and Analysis. Normalized Edit Distance: As shown in Figure 6, the distribution of edit distances reveals a concentration around 0.15–0.25. The green solid and orange dashed lines indicate the mean 0.207 and median 0.200, respectively. This concentration within the 0.1 to 0.3 range suggests that hallucinated packages are not random strings but structured “near-misses” that closely mimic the naming conventions of the npm ecosystem.

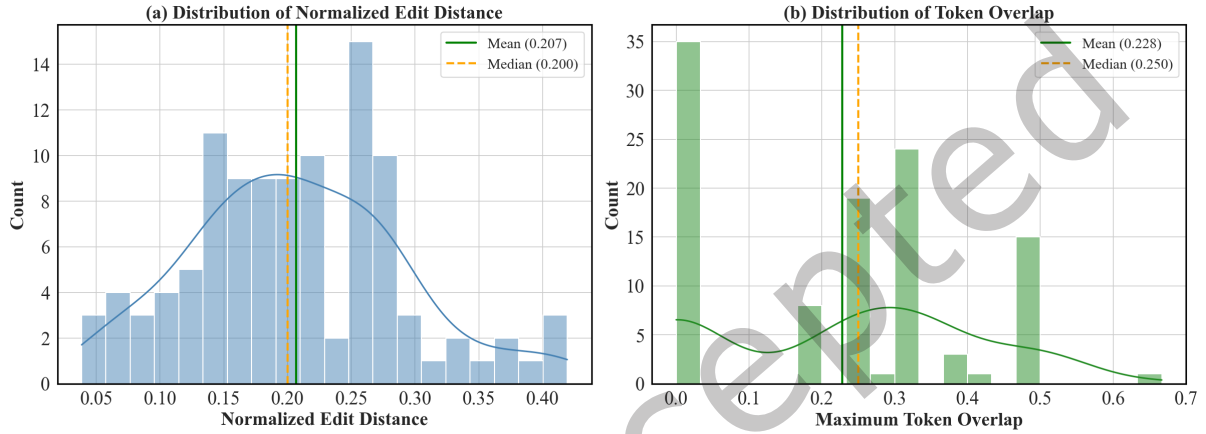


Fig. 6. Distribution of Normalized Edit Distances/Token Overlap between Hallucinated Packages

To assess whether these near-misses constitute actionable typosquatting risks, we evaluated the absolute edit distances. Our analysis reveals that 27 of the 107 hallucinated packages, or 25.2%, fall with an absolute edit distance of ≤ 1 to closest npm match. Furthermore, **49 packages** comprising **45.8%** have an absolute edit distance of ≤ 2 . The mean absolute edit distance across all 107 packages is 3.23 (median: 3), with an average package name length of 15.0 characters. Table 5 presents representative near-miss pairs spanning different edit distances and package categories.

Notably, the majority of packages in the ≤ 1 zone are Cross-Ecosystem types (e.g., pydantic→pedantic), where LLMs directly transplant a well-known name from another ecosystem that happens to be one edit away from an existing npm package. However, some are Hybrid types involving character-level permutations of real npm names (e.g., cordova-plugin-permissions→cordova-plugin-permission, vite-plugin-remove→vite-plugin-remote), which constitutes a classic typosquatting threat.

Token Overlap: As shown in Figure 6, the mean Jaccard similarity is 0.228, with only one package exhibiting an overlap exceeding 0.5. This suggests that while hallucinated packages are characteristically similar to existing packages, models typically do not merely recombine exact token sets from existing packages, but rather introduce character-level deviations or semantically related identifiers.

Naming Pattern Classification: To understand whether these targets stem from common naming conventions, we analyzed the structural patterns within our classification scheme, as detailed in Table 6. Overall, 15 of 107 packages match at least one of the six structural naming patterns, while the remaining 92 do not conform to any predefined convention. This indicates that the majority of hallucinated names do not rely on formulaic templates but instead adopt free-form naming.

Table 5. Representative Near-Miss Pairs (Sorted by Absolute Edit Distance)

Hallucinated Package	Closest npm Match	Category	Edit Distance (Abs. / Norm.)
cordova-plugin-permissions	cordova-plugin-permission	H	1 / 0.038
vite-plugin-remove	vite-plugin-remote	H	1 / 0.056
apache-httpclient	apache-httpclient1	C	1 / 0.056
core-foundation	core-foundation2	C	1 / 0.062
pydantic	pedantic	C	1 / 0.125
jasmine-marbers	jasmine-marbles	H	2 / 0.133
openssl-node	openssl-nodejs	H	2 / 0.143
neo4j-apoc	neo4j-api	C	2 / 0.200
make-plural-rules	fast-plural-rules	H	3 / 0.176
pdfplumber	plumber	C	3 / 0.300

Table 6. Naming Pattern Analysis by Package Category

Pattern	Cross-Ecosystem	Hybrid	Generic	Placeholder
Plugin Suffix	0 (0.0%)	6 (14.6%)	1 (10.0%)	2 (50.0%)
Util Suffix	0 (0.0%)	1 (2.4%)	0 (0.0%)	0 (0.0%)
API Suffix	1 (2.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
Node Prefix	0 (0.0%)	1 (2.4%)	0 (0.0%)	0 (0.0%)
JS Suffix	0 (0.0%)	1 (2.4%)	0 (0.0%)	0 (0.0%)
Template/Placeholder	0 (0.0%)	0 (0.0%)	0 (0.0%)	2 (50.0%)

Among the 15 pattern-matching packages, we observe that models frequently lean on established ecosystem conventions when fabricating names within the Hybrid and Placeholder categories. For instance, the `-plugin` suffix is utilized in 14.6% of Hybrid packages and 50% of Placeholder packages, demonstrating that models actively deploy familiar structural templates to improve the plausibility of non-existent packages. However, for Cross-Ecosystem packages, LLMs rely directly on the brand familiarity of the hijacked name rather than modifying it with npm-specific suffixes, further confirming that they construct believable hallucinations combining both brand memorization and logical structural assembly.

RQ4 Summary: LLMs generate highly plausible “near-misses” with a mean normalized edit distance of 0.207, where 25.2%, or 27 out of 107, fall in the classic typosquatting risk zone of an absolute edit distance ≤ 1 , creating exploitable risks in the software supply chain.

5.5 Hallucination Persistence Under Tool-Augmented Conditions (RQ5)

To evaluate the effectiveness of tool augmentation, we traced the origins of the 107 identified hallucinated packages. By pinpointing the specific prompt and model combinations responsible, we established a baseline of over 100 unique trigger pairs for analysis. After deduplicating the prompts, we identified 91 unique prompts. We then conducted a progressive tool-augmented experiment by feeding these 91 prompts to all 8 LLMs under three conditions of strictly increasing capability. For each condition, every prompt was tested against each model with 10 independent repetitions. This resulted in $91 \times 8 \times 10 = 7,280$ trials per condition, making 21,840 trials in total:

- **C0 (Vanilla):** Identical to the original experimental setup, with no additional tooling or prompt modifications;
- **C1 (Prompt Enhancement):** The system prompt is appended with: “Before recommending any npm package, verify it actually exists on the npm registry.”;
- **C2 (Web Search):** We enable web search through the Exa plugin provided by OpenRouter[37]. When a request is sent, the Exa engine first performs a real-time web search based on the prompt content. The search result is injected into the input as additional context. The model then generates its response based on both the original prompt and the retrieved search result.

The design logic is as follows: if hallucinations arise from a lack of verification awareness, C1 should suffice; if the model requires external information to verify package existence, C2 should be effective. By comparing hallucination rates across C0, C1, and C2, we can isolate the impact of tool augmentation on hallucination persistence.

To facilitate interpretation, we report results at two complementary levels of granularity: (1) **Pair level.** The 10 repetitions of each prompt–model pair are aggregated by majority vote: a pair is classified as “persisting” if hallucinated packages appear in ≥ 6 of 10 trials. *Pers.*, *Elim.*, and *+New* in Table 7 operate at this level. (2) **Name level.** The count of *distinct* hallucinated package names that persist across all models under a given condition.

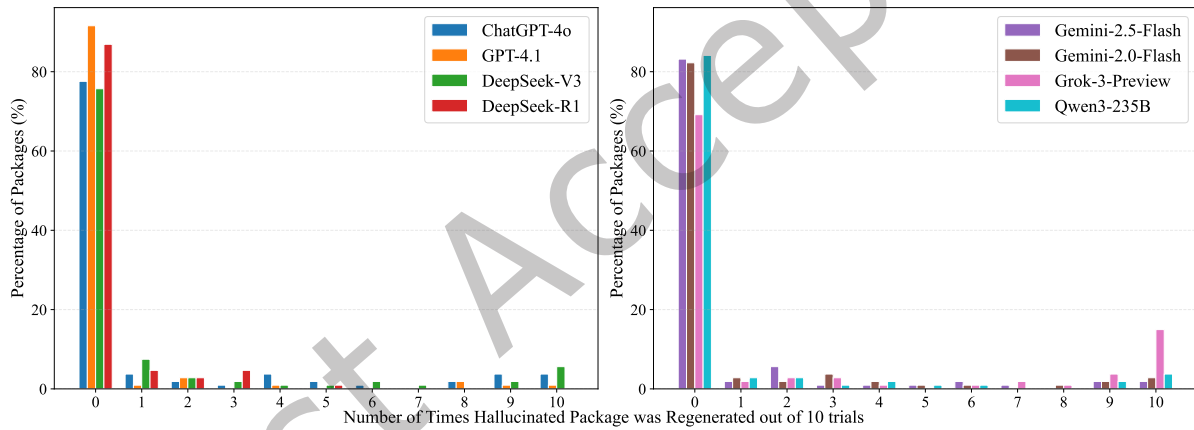


Fig. 7. Frequency of An Identical Hallucinated Package Generated From The Same Prompt across 10 Trials.

5.5.1 C0 Baseline Analysis. Before applying interventions, we first evaluated how often each hallucinated package reappears under the C0 condition. Figure 7 shows the frequency distribution of 107 hallucinated packages across 10 repeated trials for each model. The x-axis indicates how many times a specific package was regenerated, and the y-axis shows the percentage of packages at each frequency. Several patterns emerge from this baseline analysis. First, the majority of hallucinated packages are rarely regenerated. Most models show a dominant peak at zero, 66 hallucinated packages never reappear in subsequent trials. Second, a small subset of packages exhibits strong persistence. For instance, Grok-3 and DeepSeek-V3 each have packages that were regenerated in all 10 trials, suggesting certain hallucinations are deeply entrenched for specific models. Third, model behavior varies significantly. DeepSeek-R1 shows the most stable output with 93% of hallucinated packages never recurring, while Grok-3 exhibits the highest variability with only 69% of packages disappearing after the first occurrence.

This baseline analysis reveals that hallucination persistence is not uniform. While most erroneous recommendations are sporadic, a minority demonstrates systematic repetition, which motivates our investigation into whether tool augmentation can disrupt these persistent patterns.

5.5.2 Impact of Tool Augmentation. Contrary to expectations, introducing external tools does not reduce hallucination persistence but instead increases it, as shown in Figure 8(a). The overall persistence rate rises from 11.2% under the default C0 setting to 11.8% with the C1 prompt enhancement, and reaches 17.3% using the C2 web search. We grouped the prompt and model pairs and ran a McNemar test[31] to see if the tool augmentations caused a significant shift in hallucination rates for the same pairs. The full mathematical details are provided in Appendix E.

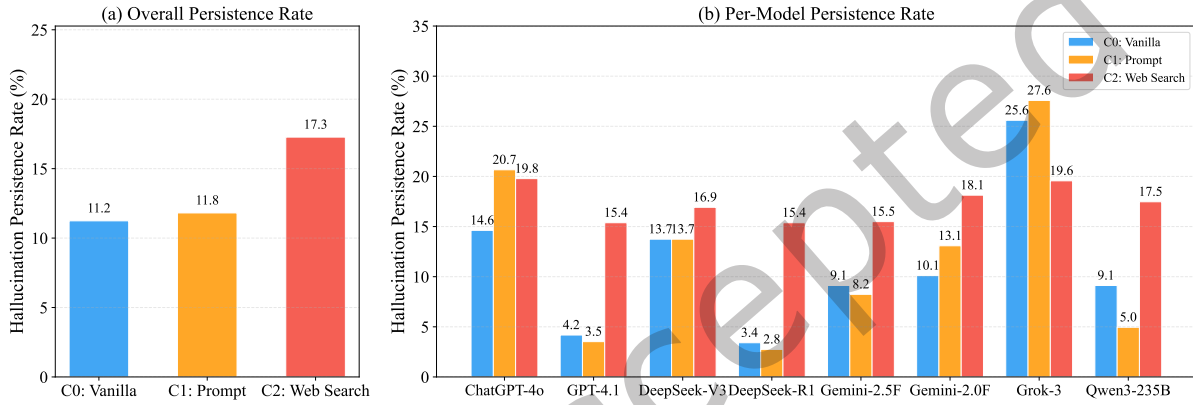


Fig. 8. Impact of Tool Augmentation on Package Hallucination Persistence. (a) Overall: Hallucination Persistence Rate Increases from C0 to C2. (b) Per-model: Individual Model Persistence Rates under Each Condition.

The results show that the increase from C0 to C1 is not statistically significant, with $\chi^2 = 2.86$ and $p = 0.091$. However, the increase from C0 to C2 is highly significant, with $\chi^2 = 33.35$ and $p < 0.001$. Within our evaluated configuration, web search augmentation amplifies rather than corrects hallucinations in the observed trials.

Pair-level dynamics. To understand why the overall rate increased, we examine how individual prompt-model pairs shifted between conditions, as shown in Table 7. Of the 69 pairs that persisted under C0, C1 eliminated only 12, or 17.4%, while introducing 23 new ones, yielding a net increase of 11 and raising the total to $69 - 12 + 23 = 80$. C2 performed even worse: it eliminated 13 pairs, or 18.8%, but introduced 65 entirely new persisting pairs, for a net increase of 52 and a total of $69 - 13 + 65 = 121$. In both conditions, the tools eliminated only a handful of existing persisting pairs while causing a much larger number of previously non-persisting pairs to begin hallucinating consistently.

Name-level analysis. Critically, this growth in persisting pairs did not reflect the invention of novel hallucinated names. As shown in Table 7, the count of unique hallucinated package names was 41 under C0, 41 under C1, and only 37 under C2. C1 produced the exact same 41 distinct names as C0, but these names appeared in 80 pairs instead of the original 69. It confirms that the additional pairs involved the same hallucinated names appearing in new model-prompt combinations. C2 exhibited an even more striking pattern: despite producing nearly twice as many persisting pairs : 121 vs 69, it actually yielded fewer distinct names 37 vs 41. In other words, tool augmentation did not generate new hallucinated packages; rather, it caused the same set of hallucination targets to persist across a broader range of model-prompt combinations that had previously shown no persistence.

Table 7. Per-Model Hallucination Persistence Rates and Pair Dynamics Under Tool-Augmented Conditions

Model	Persistence Rate (%)			C0	C0→C1 (Prompt)		C0→C2 (Web Search)	
	C0	C1	C2	Pers.	Elim.	+New	Elim.	+New
ChatGPT-4o	14.6	20.7	19.8	11	0	8	0	7
GPT-4.1	4.2	3.5	15.4	3	1	0	0	10
DeepSeek-V3	13.7	13.7	16.9	11	5	5	2	6
DeepSeek-R1	3.4	2.8	15.4	0	–	0	–	12
Gemini-2.5-Flash	9.1	8.2	15.5	7	1	2	0	7
Gemini-2.0-Flash	10.1	13.1	18.1	8	2	4	3	10
Grok-3-Preview	25.6	27.6	19.6	23	0	4	8	2
Qwen3-235B	9.1	5.0	17.5	6	3	0	0	11
Overall	11.2	11.8	17.3	69	12	23	13	65
Unique Pkg Names[†]	41	41	37					

Note. Persistence Rate is the percentage of individual trials (910 per model, 7,280 overall) in which a hallucinated package reappears.

[†]Number of distinct hallucinated package names observed across all models under each condition.

• **Finding 5: In our experiments, web search augmentation spreads existing hallucinations rather than creating new ones.** The two-level analysis converges on a single conclusion: tool augmentation propagates existing hallucinated names to new model–prompt combinations rather than inventing novel ones. The evidence consistently shows that hallucinations are simply spreading to a broader range of combinations. We can see this across two metrics: persistence rates rise from 11.2% to 17.3%, pair-level totals nearly double from 69 to 121, yet the number of unique names actually drops from 41 to 37. We hypothesize one mechanistic driver: *Context Contamination*, whereby web search injects content about real packages from other ecosystems that LLMs then recommend as npm packages.

Empirical evidence for Context Contamination. Our C2 experiment logs include the search_annotations (URL citations returned by the search engine) for each trial. 99.9% of the 1,257 C2 trials in which a hallucination persisted carried at least one annotation. Among the 65 pairs that persist only under C2 (not under C0), 18 of the 26 distinct hallucinated package names are cross-ecosystem identifiers—real packages in Python, R, Java, or C that do not exist on npm. This directly implicates the search context: the web results reference these packages by name, and the models transplant them into npm recommendations. A striking example is *undetected-chromedriver*, a well-known Python library. Under C0, 7 of 8 models produced this hallucination in $\leq 2/10$ trials; under C2, all 7 models jumped to $\geq 7/10$ persistence (e.g., GPT-4.1: 0/10→10/10; DeepSeek-R1: 2/10→10/10). The search annotations for every affected model consistently cite GitHub issues from the Python repository *ultrafunkamsterdam/undetected-chromedriver*, confirming that the injected web context reinforced the hallucinated name.

RQ5 Summary: In our evaluated setting, the web search fails to mitigate package hallucinations: it raised the persistence rate from 11.2% to 17.3%. Analysis of search annotations confirms that the amplification is driven by context contamination, where web results reference real packages from other ecosystems.

5.6 Attack Cost and Viability Analysis (RQ6)

This subsection addresses RQ6 by analyzing the cost structure of the entire attack kill chain in npm. The analysis demonstrates the attack’s high feasibility, which is characterized by its exceptionally low total cost (approximately \$496 in our configuration) and high scalability. We break down this cost structure into two primary components:

- (1) **Phase 1 & 2: Low Initial Investment Costs (Reconnaissance and Weaponization).** Startup costs are mainly in the reconnaissance (finding hallucinated packages) and weaponization (registering packages) phases.
 - **Reconnaissance Costs (Main Costs):** This is the only phase where investment is needed. Attackers must query LLM APIs on a large scale to “mine” hallucinated packages. In our experiments, querying 10,893 Stack Overflow prompts using 8 commercial LLM models cost **\$496** (as shown in Appendix F).
 - **Development Costs (One-Time Costs):** Automated scripts for querying, registering, and uploading can be written quickly by a skilled attacker, with negligible one-time effort.
 - **Registration Costs (Almost Zero):** Platforms like npm allow free registration and package publishing, enabling mass weaponization of discovered packages at no cost.
- (2) **Phase 3: Zero Marginal Cost for Malicious Delivery (Propagation).** Distribution costs are zero. Attackers rely on LLM recommendations to trigger installations, while repositories handle storage and bandwidth. Each installation has no marginal cost, giving attacks high scalability.

The total cost of the entire attack chain is only about \$496. Traditional attacks, such as zero-day vulnerability research [9, 32, 49], complex exploit chain development, and C2 infrastructure maintenance, require high expertise and costs from tens of thousands to millions of dollars. By comparison, hallucinated package attacks may have a lower entry barrier in the observed npm setting.

RQ6 Summary: In our npm deployment, hallucinated package attacks are highly feasible due to their unique features (such as zero marginal costs) and extremely low total cost (only \$496), far below traditional supply chain attacks, yet they can leverage LLM trust for large-scale poisoning.

6 NymGuard: An Automated Agent for Countering Package Hallucinations

This section details the design of NymGuard. It is designed to proactively mitigate security risks from hallucinated packages by integrating three key functions: real-time detection, preemptive package registration and dynamic early warning. This approach aims to counter supply chain attacks that exploit hallucinated packages.

6.1 Scope and Assumptions

NymGuard is evaluated as a prototype defense under controlled conditions. Its effectiveness depends on three assumptions. First, package names must be observable to the system, either from monitored LLM outputs, public reports, prompt-mining workflows or registry-side discovery pipelines. Second, the package registry API must be reachable and sufficiently up-to-date at the time of verification. Third, the defense assumes responsible custodianship of placeholder packages, including transparent disclosure and a process for releasing a name to legitimate maintainers.

6.2 NymGuard Modules

The architecture of NymGuard is structured around three core modules: the Hallucination Detection Module, the Package Squatting Module and the Real-time Warning Module. The interaction between these modules forms a complete operational workflow, which is visually depicted in Figure 9.

6.2.1 Hallucination Detection Module. This module is responsible for identifying hallucinated package names in code generated by LLMs. It continuously monitors LLM outputs, extracts package names and verifies their existence in official registries. The process begins by using regular expressions to extract package names from installation commands (e.g., `npm install "package-name"`). Subsequently, the module queries the API of the relevant official registry to determine if the extracted name is registered. Package names are classified as package hallucinations if the registry does not contain them, which serves as a definitive detection method.

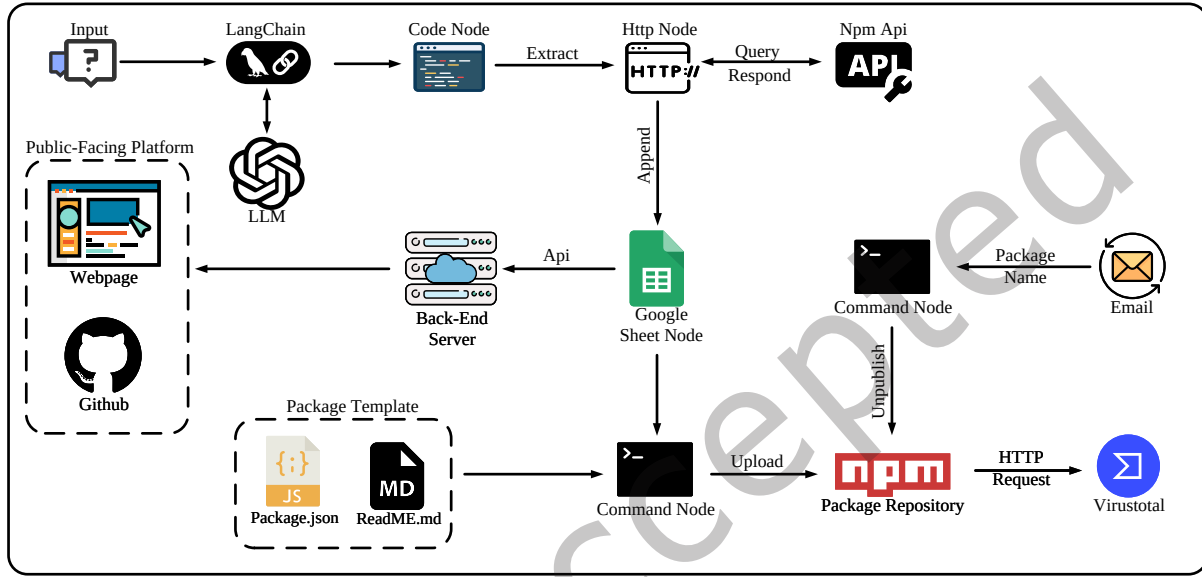


Fig. 9. The NymGuard Workflow

6.2.2 Dynamic Package Squatting Module. Once a hallucinated package is identified, this module executes a preemptive defense strategy. Its workflow is divided into two phases:

- **Phase 1: Preemptive Registration and Custodianship** Initially, the module automatically registers a placeholder package under the hallucinated name in npm. This action immediately prevents malicious actors from exploiting the name, neutralizing a potential supply chain attack vector. The placeholder package contains a `package.json` file and a `README.md` file. The `package.json` specifies the package name, a security-focused version (e.g., “0.0.1-security”) and a description indicating it is a security placeholder. The `README.md` file indicates that this package is a benign placeholder, registered by our research group to prevent potential misuse, and provides a specific contact email.

- **Phase 2: Automated Release and Continuous Monitoring** This phase ensures that package names can be responsibly returned to the community. NymGuard continuously monitors the designated contact channel. Upon receiving a legitimate request from a developer, the agent automatically validates it and executes “`npm unpublish`” command to release the package name. The package name is added to a high-priority watchlist. NymGuard then actively monitors the status in npm. If a new version is published under this name, the system immediately downloads it for in-depth analysis by a security engine such as VirusTotal.

6.2.3 Real-time Warning Module. This module is designed for transparent disclosure to the developer and security communities. This mechanism operates through a public GitHub repository and a dedicated project website, which

hosts the central information hub. The core of this hub is a dynamic dashboard that functions as a real-time early warning system for package hallucinations. For each entry, the dashboard provides a comprehensive overview, detailing the package name, the timestamp of its registration, the source LLM, a risk level and a hyperlink to the benign placeholder on the npm registry. The content of the dashboard is populated automatically, ensuring that security advisories are made public concurrently with the execution of each preemptive action.

6.3 Effectiveness Evaluation

This subsection systematically evaluates the performance of NymGuard in terms of its hallucinated package discovery efficiency and benign preemption performance.

6.3.1 Evaluation of Hallucinated Package Verification. The hallucination detection module employs a deterministic verification process. It validates potential package names by querying the official registry API. A definitive “404” response serves as ground-truth confirmation that the package is nonexistent. The ability to effectively find verifiable hallucinated packages from LLM outputs is deterministic, which guarantees a verification accuracy of 100%.

6.3.2 Efficiency and Stability of the Preemption Process. The experiment is designed to precisely measure the end-to-end response time of NymGuard in preempting hallucinated packages and to evaluate the overall success rate of the process. We defined the response time as the interval from the initial API verification confirming the nonexistence of package names (T_{start}) to a successful placeholder publication (T_{end}). The response time is calculated as $T_{\text{response}} = T_{\text{end}} - T_{\text{start}}$. A list of 26 verified and unregistered package names was utilized. NymGuard then sequentially executed a preemption workflow for these packages. The key performance indicators are summarized in Table 8, which shows that NymGuard exhibits second-level responsiveness and a high success rate.

Table 8. Summary of NymGuard Performance (N=26)

Performance Metric	Value
End-to-End Response Time (s)	–
Median	6.534
Mean $\pm$ Std. Dev.	6.583 $\pm$ 0.232
Range [Min, Max]	[6.278, 7.551]
Task Outcomes & Robustness	–
Success Rate	81%(21/26)
Failure Reasons (n)	API Error (4), Network Timeout (1)

6.3.3 Retrospective Temporal Validation. We provide a retrospective temporal analysis using our six-month longitudinal data to quantify NymGuard’s protective potential. Our telemetry data reveals that the fastest observed first developer installation occurred merely **8 minutes** after package registration, when a US-based developer IP downloaded the *pdfplumber* package. Even under this most aggressive scenario, NymGuard’s median preemption latency of 6.534 seconds is approximately **73× faster** than the earliest observed developer exposure, providing a substantial safety margin. Across all 40 packages that received non-automated traffic, the median time-to-first-install was approximately 12 days, making NymGuard over 150,000× faster in the typical case. Table 9 summarizes this comparison.

Critically, even in the worst case where a developer attempts installation within minutes of publication, NymGuard’s second-level response time ensures that the namespace is already occupied by a safe placeholder

Table 9. Retrospective Comparison: Observed Exposure vs. NymGuard Preemption

Metric	Without NymGuard	With NymGuard
Time to namespace occupation	Attacker-dependent	6.5 s (median)
Fastest developer exposure	8 min (observed)	0 (preempted)
Median developer exposure	12 days	0 days (preempted)
Packages receiving dev. installs	40+	0 (safe placeholder)
Total developer IPs exposed	121	0

before any human interaction can occur. Furthermore, this preemptive mechanism mirrors the established security practice of the npm security team itself: when npm identifies and removes a malicious package, they publish a security placeholder (typically versioned 0.0.1-security) under the same name to prevent re-registration.

6.4 NymGuard Deployment and Dissemination

6.4.1 NymGuard Deployment. NymGuard is deployed in a containerized n8n [33] environment using Docker, ensuring portability, isolation and operational stability. The functions, such as detection and preemption, are designed as n8n workflows to facilitate sharing and reuse. Within this environment, API credentials, configurations, and data persistence are managed securely and centrally, supporting long-term operation and performance evaluation.

6.4.2 Dissemination. NymGuard features two public-facing platforms designed for community engagement and the dissemination of results: a GitHub repository and a dedicated project website. The repository, located at <https://github.com/xiaoye798/Package-Hallucination-Research>, hosts the complete source code, architectural documentation and the list of identified hallucinated packages to ensure transparency and reproducibility. These platforms serve a dual purpose: first, they offer an immediate defense tool for software developers; second, they provide the research community with a valuable dataset on package hallucinations to support further analysis of this security challenge.

7 Discussion

7.1 The Necessity of Downstream Defense

While reducing hallucinations directly within LLMs is the ideal root-cause solution, it faces major practical limits. The code-generation landscape is largely dominated by closed-source/black-box models (e.g., Claude 4.6 Opus, GPT-5.3 Codex). Because current LLMs make their outputs inherently unpredictable and fundamentally uncontrollable, relying solely on upstream resolution leaves the OSS supply chain highly risk during the indefinite period. Therefore, parallel mitigation is required; tools like NymGuard are not a displacement of responsibility, but a critical defense-in-depth necessity. They empower the downstream ecosystem to build an active self-preservation mechanism against unpredictable and uncontrollable upstream inputs.

7.2 Economic Impact and the “Super-Spreader” Phenomenon

The economic analysis (RQ6) highlights a striking imbalance in the npm setting we observed. For a low search cost of approximately **\$496**, an attacker can prepare tens of thousands of hallucinated packages for an attack. This low barrier to entry, combined with distribution at no extra cost (where LLMs act as an unwitting recommender), makes this a scalable and widely available threat. It is far more accessible than traditional vulnerability research.

We also observed a pronounced concentration of adoption: among 107 packages, only two packages attracted substantial developer interactions (RQ3). While two cases preclude statistical generalization, both packages share characteristics that may be indicative of higher risk: cross-ecosystem brand recognition, high task-domain relevance in the target language. Future work with larger-scale deployments is needed to validate whether these characteristics constitute a reliable risk profile. In the interim, these observations suggest a preliminary direction for risk-aware monitoring. For instance, prioritizing the preemption of names that combine cross-ecosystem brand recognition with task-domain plausibility may be warranted, but such heuristics should be treated as tentative guidance.

7.3 Implications for the Software Supply Chain

Our findings have important implications for all groups in the software supply chain.

For LLM Providers: Models are no longer neutral code generators; they are active participants in security. As LLMs integrate more into developer tools like IDEs, they act as powerful but imperfect recommenders. In light of this reality, the responsibility shifts to providers, requiring real-time checks of package registries. NymGuard automates this process as a key mitigation. They could also use other pre-generation techniques to prevent fabricated recommendations.

For Repository Managers (e.g., npm): The 12-day survival window we saw for many packages shows a serious gap in security. The current security posture is mostly reactive. This gives attackers a predictable window to exploit. This delay is not enough to counter a fast-moving, LLM-driven attack. Active defenses are becoming essential. These include monitoring for and preemptively registering names that fit a high-risk profile.

7.4 Ecosystem Impact and Sustainability

A potential concern regarding proactive preemption is whether an automated registration strategy might pollute the package registry index or waste infrastructure resources. First, hallucinated packages are not randomly or infinitely generated. They are remarkably finite and highly repetitive. As observed in large-scale evaluation, querying 10,893 developer prompts across eight models yielded only 107 unique hallucinated packages. Registering a highly bounded set of placeholders introduces practically zero index pollution to the npm registry, which currently hosts over 3.8 million packages. Second, the resource footprint of NymGuard is microscopic. The placeholder packages consist solely of a lightweight *package.json* and a *README.md* file. They contain no executable code, no binaries and zero downstream dependencies. Consequently, the total uncompressed size of a single placeholder is typically under 5 KB. Even if all 107 hallucinated packages were preemptively registered, the cumulative storage footprint would be less than 1 MB. For massive cloud infrastructures like npm, the bandwidth and storage consumed by these static text files are essentially zero, meaning no resources are wasted.

7.5 Limitations

NymGuard is a prototype downstream defense evaluated under controlled conditions, and its claims should be interpreted within this scope. We discuss three categories of limitations below.

Failure Cases. NymGuard achieved an 81% (21/26) preemption success rate in our prototype evaluation, with failures caused by external API errors and network timeouts. Therefore, this success rate reflects a prototype baseline rather than a comprehensive security guarantee.

Adversarial Assumptions. NymGuard can be bypassed under specific adversarial scenarios. It cannot protect against attackers who register hallucinated names before the system detects them. Thus, NymGuard aims to reduce opportunistic attack surfaces rather than completely stopping sophisticated, targeted adversaries.

Operational Cost. The long-term sustainability of NymGuard depends on its deployment scale. In bounded deployments, the primary expense is LLM API usage for prompt mining (approximately \$496 for 10,893 prompts

in our study), while infrastructure and registry fees are negligible. However, our six-month evaluation does not capture extended management costs, such as abuse handling and user governance. Consequently, open-ended global monitoring is beyond the scope of this prototype and would require broad collaboration across the ecosystem.

8 Conclusion

This paper demonstrates the practical feasibility of a potential attack vector in which LLM-generated package hallucinations can be registered and exploited in the npm ecosystem. To mitigate this risk, we developed and open-sourced NymGuard for proactive detection and pre-emptive registration. As LLM-driven tools become more integrated into software development, our work serves as a critical call to action for a multi-layered defense.

Acknowledgments

This work is supported by Zhongguancun Laboratory and the National Cyber Security–National Science and Technology Major Project 2025ZD1502100.

References

- [1] Ricardo Amaro, Rúben Pereira, and Miguel Mira da Silva. 2025. Mapping DevOps Capabilities to the Software Life Cycle: A Systematic Literature Review. *Information and Software Technology* 177 (2025), 107583.
- [2] Sunil Anasuri and Guru Pramod Rusum. 2024. Software Supply Chain Security: Policy, Tooling, and Real-World Incidents. *International Journal of Emerging Trends in Computer Science and Information Technology* 5, 3 (2024), 79–89.
- [3] Fraol Batole, David OBrien, Tien N. Nguyen, Robert Dyer, and Hridesh Rajan. 2025. An LLM-Based Agent-Oriented Approach for Automated Code Design Issue Localization. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1320–1332. doi:10.1109/ICSE55347.2025.00100
- [4] Giacomo Benedetti, Oreofe Solarin, Courtney Miller, Greg Tystahl, William Enck, Christian Kastner, Alexandros Kapravelos, Alessio Merlo, and Luca Verderame. 2025. An Empirical Study on Reproducible Packaging in Open-Source Ecosystems. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1052–1063. doi:10.1109/ICSE55347.2025.00136
- [5] Tingting Bi, Boming Xia, Zhenchang Xing, Qinghua Lu, and Liming Zhu. 2024. On The Way to Sboms: Investigating Design Issues and Solutions in Practice. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–25.
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-shot Learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [7] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael Jordan, Joseph E Gonzalez, et al. 2024. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. In *Forty-first International Conference on Machine Learning*. JMLR.org, 30 pages.
- [8] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and psychological measurement* 20, 1 (1960), 37–46.
- [9] Zhen Yu Ding and Claire Le Goues. 2021. An Empirical Study of OSS-Fuzz Bugs. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE Computer Society, Los Alamitos, CA, USA, 131–142. doi:10.1109/MSR52588.2021.00026
- [10] Charlie Eriksen. 2025. npm debug and chalk packages compromised. <https://www.aikido.dev/blog/npm-debug-and-chalk-packages-compromised>. [Online; accessed: 29-Oct-2025].
- [11] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models (*KDD '24*). Association for Computing Machinery, New York, NY, USA, 6491–6501. doi:10.1145/3637528.3671470
- [12] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Asmita, Ryan Tsang, Najmeh Nazari, Han Wang, and Houman Homayoun. 2024. Large Language Models for Code Analysis: Do LLMs Really Do Their Job?. In *Proceedings of the 33rd USENIX Conference on Security Symposium (Philadelphia, PA, USA) (SEC '24)*. USENIX Association, USA, Article 47, 18 pages.
- [13] Md Jobair Hossain Faruk, Masrura Tasnim, Hossain Shahriar, Maria Valero, Akond Rahman, and Fan Wu. 2022. Investigating Novel Approaches to Defend Software Supply Chain Attacks. In *2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 283–288.
- [14] Google Cloud. 2025. Software Supply Chain Security: Attack Vectors. <https://docs.cloud.google.com/software-supply-chain-security/docs/attack-vectors>. [Online; accessed: 31-Oct-2025].

- [15] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
- [16] Yinghao Hu, Leilei Gan, Wenyi Xiao, Kun Kuang, and Fei Wu. 2025. Fine-tuning Large Language Models for Improving Factuality in Legal Question Answering. *arXiv preprint arXiv:2501.06521* (2025).
- [17] Cheng Huang, Nannan Wang, Ziyang Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. 2024. {DONAPI}: Malicious {NPM} Packages Detector Using Behavior Sequence Knowledge Mapping. In *33rd USENIX Security Symposium (USENIX Security 24)*. 3765–3782.
- [18] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55.
- [19] Qing Huang, Zhiwen Luo, Zhenchang Xing, Jinshan Zeng, Jieshan Chen, Xiwei Xu, and Yong Chen. 2024. Revealing the Unseen: AI Chain on LLMs for Predicting Implicit Dataflows to Generate Dataflow Graphs in Dynamically Typed Code. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–29.
- [20] Saki Imai. 2022. Is GitHub Copilot a Substitute for Human Pair-programming? An Empirical Study. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE Computer Society, Los Alamitos, CA, USA, 319–321. doi:10.1109/ICSE-Companion55297.2022.9793778
- [21] Nihal Jain, Robert Kwiatkowski, Baishakhi Ray, Murali Krishna Ramanathan, and Varun Kumar. 2025. On Mitigating Code LLM Hallucinations with API Documentation. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 237–248.
- [22] Arjun Krishna, Erick Galinkin, Leon Derczynski, and Jeffrey Martin. 2025. Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities. *arXiv preprint arXiv:2501.19012* (2025).
- [23] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 1509–1526. doi:10.1109/SP46215.2023.10179304
- [24] Enrique Larios Vargas, Mauricio Aniche, Christoph Treude, Magiel Bruntink, and Georgios Gousios. 2020. Selecting Third-party Libraries: The Practitioners’ Perspective. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 245–256.
- [25] Jasmine Latendresse, Naoures Day, SayedHassan Khatoonabadi, and Emad Shihab. 2025. The Software Librarian: Python Package Insights for Copilot. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE Computer Society, Los Alamitos, CA, USA, 17–20. doi:10.1109/ICSE-Companion66252.2025.00014
- [26] Vladimir I. Levenshtein. 1966. Binary Codes Capable of Correcting Deletions, Insertions, and Reversals. *Soviet Physics Doklady* 10, 8 (1966), 707–710.
- [27] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented Generation for Knowledge-intensive Nlp Tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [28] Junyi Li, Jie Chen, Ruiyang Ren, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2024. The Dawn After the Dark: An Empirical Study on Factuality Hallucination in Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- [29] X. Liu, Y. Liu, Y. Zhang, J. Li, and S. Hu. 2026. PackMonitor: Enabling Zero Package Hallucinations Through Decoding-Time Monitoring. In *Proceedings of the 35th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.
- [30] Changhua Luo, Wei Meng, and Shuai Wang. 2024. Strengthening Supply Chain Security with Fine-grained Safe Patch Identification. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.
- [31] Quinn McNemar. 1947. Note on The Sampling Error of The Difference between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157.
- [32] Ruijie Meng, Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. 2025. AFLNet Five Years Later: On Coverage-Guided Protocol Fuzzing. *IEEE Transactions on Software Engineering* (2025).
- [33] n8n.io. 2019. n8n: Fair-code workflow automation platform. <https://github.com/n8n-io/n8n>. [Online; accessed: 29-Oct-2025].
- [34] Andrew Nesbitt. 2015. Libraries.io: Open Source Software Dependencies and Metadata. <https://libraries.io>. [Online; accessed: 25-Feb-2026].
- [35] Shradha Neupane, Grant Holmes, Elizabeth Wyss, Drew Davidson, and Lorenzo De Carli. 2023. Beyond Typosquatting: an In-depth Look at Package Confusion. In *Proceedings of the 32nd USENIX Conference on Security Symposium (Anaheim, CA, USA) (SEC ’23)*. USENIX Association, USA, Article 193, 18 pages.
- [36] Npm. 2025. Npm CLI: Command Line Interface for npm. <https://github.com/npm/cli>. [Online; accessed: 4-June-2025].

- [37] OpenRouter. 2024. Web Search | Add Real-time Web Data to AI Model Responses. <https://openrouter.ai/docs/guides/features/plugins/web-search>. [Online; accessed: 10-Mar-2026].
- [38] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at The Keyboard? Assessing the Security of Github Copilot's Code Contributions. *Commun. ACM* 68, 2 (2025), 96–105.
- [39] Huilian Sophie Qiu, Zihe H Zhao, Tielin Katy Yu, Justin Wang, Alexander Ma, Hongbo Fang, Laura Dabbish, and Bogdan Vasilescu. 2023. Gender Representation among Contributors to Open-source Infrastructure: an Analysis of 20 Package Manager Ecosystems. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*. IEEE, 180–187.
- [40] Jukka Ruohonen and Mubashrah Saddiq. 2025. A Time Series Analysis of Malware Uploads to Programming Language Ecosystems. In *International Conference on Availability, Reliability and Security*. Springer, 269–285.
- [41] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at C: A User Study on The Security Implications of Large Language Model Code Assistants. In *Proceedings of the 32nd USENIX Conference on Security Symposium (Anaheim, CA, USA) (SEC '23)*. USENIX Association, USA, Article 124, 18 pages.
- [42] Joseph Spracklen, Raveen Wijewickrama, AHM Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. 2024. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. In *Proceedings of the 34th USENIX Security Symposium*. USENIX Association.
- [43] Stack Overflow. 2025. Stack Overflow: The World's Largest Online Community for Developers. <https://stackoverflow.com/>. [Online; accessed: 11-June-2025].
- [44] Andy Thompson. 2025. Sandworm in The Supply Chain: Lessons from The Shai-Hulud Npm Attack on Developer and Machine Identities. <https://www.cyberark.com/resources/blog/sandworm-in-the-supply-chain-lessons-from-the-shai-hulud-npm-attack-on-developer-and-machine-identities>. [Online; accessed: 29-Oct-2025].
- [45] Katherine Tian, Eric Mitchell, Huaxiu Yao, Christopher Manning, and Chelsea Finn. 2023. Fine-tuning Language Models for Factuality. In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*.
- [46] SMTI Tonmoy, SM Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. 2024. A Comprehensive Survey of Hallucination Mitigation Techniques in Large Language Models. *arXiv preprint arXiv:2401.01313* 6 (2024).
- [47] Duc-Ly Vu, Zachary Newman, and John Speed Meyers. 2023. Bad Snakes: Understanding and Improving Python Package Index Malware Scanning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 499–511.
- [48] Fanyu Wang, Chetan Arora, Chakkrit Tantithamthavorn, Kaicheng Huang, and Aldeida Aleti. 2025. Requirements-driven Automated Software Testing: A Systematic Review. *ACM Transactions on Software Engineering and Methodology* (2025).
- [49] Felix Weissberg, Jonas Möller, Tom Ganz, Erik Imgrund, Lukas Pirch, Lukas Seidel, Moritz Schloegel, Thorsten Eisenhofer, and Konrad Rieck. 2024. SoK: Where to Fuzz? Assessing Target Selection Methods in Directed Fuzzing. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (Singapore, Singapore) (ASIA CCS '24)*. Association for Computing Machinery, New York, NY, USA, 1539–1553. doi:10.1145/3634737.3661141
- [50] Laurie Williams, Giacomo Benedetti, Sivana Hamer, Ranindya Paramitha, Imranur Rahman, Mahzabin Tamanna, Greg Tystahl, Nusrat Zahan, Patrick Morrison, Yasemin Acar, et al. 2025. Research Directions in Software Supply Chain Security. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–38.
- [51] Shengqiong Wu, Hao Fei, Liangming Pan, William Yang Wang, Shuicheng Yan, and Tat-Seng Chua. 2025. Combating Multimodal LLM Hallucination via Bottom-Up Holistic Reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 8460–8468.
- [52] Shenao Yan, Shen Wang, Yue Duan, Hanbin Hong, Kiho Lee, Doowon Kim, and Yuan Hong. 2024. An LLM-assisted Easy-to-trigger Backdoor Attack on Code Completion Models: Injecting Disguised Vulnerabilities Against Strong Detection. In *Proceedings of the 33rd USENIX Conference on Security Symposium (Philadelphia, PA, USA) (SEC '24)*. USENIX Association, USA, Article 101, 18 pages.
- [53] Jun Yu, Yunxiang Zhang, Zerui Zhang, Zhao Yang, Gongpeng Zhao, Fengzhao Sun, Fanrui Zhang, Qingsong Liu, Jianqing Sun, Jiaen Liang, and Yaohui Zhang. 2024. RAG-Guided Large Language Models for Visual Spatial Description with Adaptive Hallucination Corrector. In *Proceedings of the 32nd ACM International Conference on Multimedia (Melbourne VIC, Australia) (MM '24)*. Association for Computing Machinery, New York, NY, USA, 11407–11413. doi:10.1145/3664647.3688990
- [54] Nusrat Zahan. 2023. Software Supply Chain Risk Assessment Framework. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 251–255.
- [55] Chaowei Zhang, Zongling Feng, Zewei Zhang, Jipeng Qiang, Guandong Xu, and Yun Li. 2025. Is LLMs Hallucination Usable? LLM-based Negative Reasoning for Fake News Detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 1031–1039.
- [56] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. 2025. Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI Using a Single Model of Malicious Behavior Sequence. *ACM Transactions on Software Engineering and Methodology* 34, 4 (2025), 1–28.

A Detailed IP Address Classification Rules

This section details the comprehensive rule set employed for the IP address attribution analysis discussed in Section 5.1. Table 10 enumerates the specific attributes, conditions, and observations used by our experts to construct the 100-sample ground-truth corpus. This structured guide ensured consistency during manual annotation and formed the feature basis for training the LightGBM classifier. The rules are organized into three primary categories: Static Attributes, Behavioral Characteristics, and Special Case Handling.

Table 10. The IP Address Classification Rule

Type	Category	Attribute / Condition / Observation
Static Attribute	IP Reputation	Officially marked as malicious
	Threat Type	Associated threat intelligence indicates specific keywords
	Network Context	Identified as originating from a server environment/ an end-user environment
	Cloud Services	Attributed to a Cloud Service Provider AND evidence suggests non-individual user
	ISP Type	Attributed to a conventional Internet Service Provider
	Benign Status	No known malicious markers AND no identified threat types associated with the IP
Behavioral Characteristic	Connection Patterns	Predominance of numerous, very short-duration connections Connection durations are exactly identical across multiple distinct sessions
	Operating System	Identified as a server-grade OS (e.g., Linux Server variants, Windows Server) Identified as a desktop-grade OS (e.g., Windows 10/11, macOS, common Linux desktops)
	User Account Type	Username indicative of a service account (e.g., “admin”, “service”, “system” and “runner”) Username appears personalized or non-generic
	Hostname Structure	Hostname follows a common server naming convention (e.g., ‘srv-’, ‘db-’, patterned, cloud-instance IDs) Hostname appears personalized or non-generic
	CPU Architecture	CPU model identified as server-grade (e.g., Xeon, EPYC) CPU model identified as consumer-grade (e.g., Core i-series, Ryzen)
	Display/GUI	Absence of a graphical user interface indicated (e.g., “N/A (Linux/No X)”, no screen resolution) Presence of a common desktop display resolution
	Session Interactivity	Evidence of an interactive session (e.g., real-time command execution, user-driven events) Evidence of a non-interactive or scripted session (e.g., automated task execution)
	Process Footprint	Presence of web browser processes detected Absence of web browser processes detected

Type	Category	Attribute / Condition / Observation
		Presence of office productivity software processes (e.g., Microsoft Office suite)
	Process Enumeration	Failure to retrieve or access process information
	System Uptime	Exceptionally long system uptime (e.g., continuously operational for >30 days)
	Feature Aggregation (Server)	≥ 3 distinct behavioral characteristics indicative of a server environment present
	Feature Aggregation (Desktop)	≥ 3 distinct behavioral characteristics indicative of a desktop/user environment present
Special Case Handling	VPN/Proxy Usage	Accompanied by demonstrable user-like behavioral patterns/ Lacks demonstrable user-like behavioral patterns; activity appears automated
	Cloud Service User Behavior	IP originates from a cloud service, but exhibits clear, strong user-like behaviors/ no user behavior, and current cumulative score is less than 5

Illustrative IP Classification Examples

- **Real User:**
 - **IP:** 123.45.xxx.xxx
 - **Details:** OS: Windows 10 Desktop; ISP: Residential; GUI: Present; Hostname: Personalized String; Session: Interactive with Browser Processes.
- **Automated Script:**
 - **IP:** 98.76.xxx.xxx
 - **Details:** OS: Ubuntu Server; ISP: Cloud Provider; GUI: Absent; Hostname: Auto-generated Instance ID; Session: Non-interactive, Repeated identical duration connections.
- **Weak User Signal:**
 - **IP:** 45.67.xxx.xxx
 - **Details:** OS: macOS Desktop; ISP: Corporate Network; GUI: Present; Hostname: Generic Pattern; Session: Intermittent Interaction Detected.

B Supplementary Research Methods

B.1 Method of Hallucinated Package Classification

This subsection describes the classification method for the 107 hallucinated packages, supporting the results in Section 5.3.2. The process consisted of two stages: automated matching and manual review.

(1) **Keyword and regex matching:** Build keyword lists and regular expressions based on category definitions to automatically match package names; (2) **Data source:** Brand lists collected from major ecosystems (Python PyPI, Java Maven) based on top download projects; (3) **Manual verification:** Names with ambiguity were checked manually.

B.2 Prompt Dataset Classification Details

This subsection details the semantic classification methodology for the 10,893 Stack Overflow queries (adopted from Spracklen et al. [42]) introduced in Section 4.1.

B.2.1 Annotation Method. The annotation method was automated keyword matching based on regular expression text search. The classification standards are defined below. (1) **Core Keywords:** parse, extract, read, scrape, crawl, fetch, load, import, download; (2) **Data Retrieval:** get data, data extract, retrieve data; (3) **File Formats:** HTML, PDF, XML, JSON, CSV, YAML, TXT; (4) **Operation Types:** text extract, file read, data parsing, web scraping.

B.2.2 Statistical Analysis Query Keywords. (1) **Visualization:** plot, chart, graph, visualization, dashboard; (2) **Statistical Methods:** statistic, analysis, regression, correlation; (3) **Mathematical Operations:** calculate, math, algorithm, metric; (4) **Data Science:** model, machine learning, data science, predict (5) **Report Generation:** report, analytics.

B.2.3 Classification Results. The automated classification yielded the following distribution across the 10,893 queries. The data confirms that queries related to data extraction significantly outweigh those for statistical analysis.

Table 11. Query Classification Counts (N=10,893)

Category	Query Count	Percentage	1_question.json	2_question.json
Data Extraction	6,916	63.5%	4,066	2,850
Statistical Analysis	2,704	24.8%	1,646	1,058
Other Categories	1,273	11.7%	708	565
Total	10,893	100%	5,420	5,473

C Complete List of Hallucinated Packages

This appendix contains the full list of the 107 hallucinated packages we analyzed, detailed in Table 12. To support our analysis in Section 5, we tracked seven key attributes for each package: the Package Name (the exact name we registered on npm); the Type: our assigned category as defined in Section 5.3.2; the Task Domain: its supposed purpose, e.g., “Data Processing”, used for RQ3; the Model Count: the number of LLMs out of eight that generated the name, showing how widespread it was; its Days: how long it stayed on npm before removal; Downloads: the total cumulative download count recorded by the npm registry during the package’s lifetime; and Unique IPs: the number of distinct IP addresses from which installation requests were received, as logged by our C2 callback server.

Figure 10 plots the total number of downloads against unique developer IPs for every package that was downloaded at least once. The plot reveals two distinct patterns. First, two super-spreader packages, *pdfplumber* and *beautifulsoup4*, stand out with both high download counts and high unique IPs, confirming genuine developer adoption. Second, packages such as *dayjs-plugin-duration* and *next-refresh-token* recorded 728 and 557 downloads respectively, yet each came from only a single unique IP. This indicates that their traffic is highly likely to be automated.

Table 12. Package Information Statistics (C: Cross-Ecosystem, H: Hybrid, G: Generic, P: Placeholder)

No.	Package Name	Type	Task Domain	Models/Days	Downloads/IPs
1	mongoose-to-json	H	Database & Storage	7 / 12	196 / 22

Continued on next page

Table 12 – Continued

No.	Package Name	Type	Task Domain	Models /Days	Downloads / IPs
2	pdfplumber	C	Data Processing & Analysis	5 / 32	374 / 102
3	sharp-heic	H	General Programming	3 / 12	156 / 24
4	highcharts-mouse-wheel-zoom	H	UI & Styling	2 / 12	141 / 10
5	neo4j-apoc	C	Database & Storage	4 / 12	125 / 10
6	neo4j-apoc-procedures	C	Database & Storage	2 / 12	130 / 11
7	data-memocombinators	G	Data Processing & Analysis	1 / 12	135 / 8
8	fasteners	C	General Programming	4 / 12	134 / 10
9	phpseclib	C	Security & Authentication	4 / 12	141 / 14
10	healenium-web	C	Testing & Quality	3 / 12	132 / 11
11	healenium-webdriverio	H	Testing & Quality	2 / 12	139 / 9
12	flutter_cupertino_date_picker	C	UI & Styling	2 / 12	121 / 6
13	some-error-logging-package	P	Error & Troubleshooting	1 / 12	143 / 12
14	vite-plugin-monorepo	G	Configuration & Setup	1 / 12	130 / 10
15	lme4	C	General Programming	3 / 12	135 / 11
16	unreal-engine-material-editor	H	Game Development	1 / 12	137 / 13
17	scatterpie	C	Data Processing & Analysis	7 / 12	140 / 12
18	dyalog-ride	C	Error & Troubleshooting	1 / 12	136 / 10
19	make-plural-rules	H	General Programming	1 / 14	177 / 7
20	vite-plugin-remove	H	Configuration & Setup	1 / 14	174 / 6
21	vue-highcharts-official	H	UI & Styling	1 / 14	174 / 1
22	istanbul-reporter-lcov	H	General Programming	1 / 14	183 / 1
23	vite-plugin-node-modules-polyfills	H	Configuration & Setup	1 / 14	154 / 1
24	compiletest_rs	C	Testing & Quality	1 / 14	169 / 8
25	jetty-quick	C	UI & Styling	2 / 14	180 / 1
26	haml-jst-loader	H	Data Processing & Analysis	1 / 14	171 / 1
27	your-published-package	P	General Programming	1 / 14	174 / 1

Continued on next page

Table 12 – Continued

No.	Package Name	Type	Task Domain	Models /Days	Downloads / IPs
28	kidding	G	General Programming	1 / 14	175 / 1
29	psalm	C	Error & Troubleshooting	3 / 14	179 / 1
30	shaderc	C	General Programming	7 / 14	164 / 1
31	cordova-plugin-permissions	H	Configuration & Setup	1 / 14	227 / 1
32	typescript-plugin-some-plugin	P	Configuration & Setup	1 / 14	176 / 6
33	babel-plugin-some-plugin	P	Configuration & Setup	1 / 16	166 / 1
34	jest-coverage-merge	H	Testing & Quality	1 / 14	191 / 1
35	josson	G	General Programming	3 / 14	165 / 1
36	scheme-latin-1-transcoder	G	General Programming	1 / 14	172 / 1
37	wdio-healenium-service	H	Testing & Quality	1 / 14	143 / 1
38	fanotify	G	System & Hardware	6 / 28	133 / 1
39	logdna-agent	C	Error & Troubleshooting	2 / 28	139 / 1
40	hungarian-algorithm	G	Data Processing & Analysis	2 / 14	175 / 1
41	sails-mssql-adapter	H	Database & Storage	1 / 14	165 / 1
42	jna	C	General Programming	5 / 14	170 / 1
43	meteor-roles	H	Web Development	1 / 15	174 / 4
44	seaborn	C	Data Processing & Analysis	1 / 15	296 / 1
45	swagger2-postman2	H	General Programming	1 / 15	163 / 1
46	mock-arduino	C	Testing & Quality	1 / 15	179 / 1
47	arduino-mock	C	Testing & Quality	1 / 28	143 / 1
48	slf4j-api-js	H	Web Development	1 / 15	186 / 1
49	concurrent-hashmap	G	General Programming	0 / 15	185 / 1
50	reactotron-mobx	H	UI & Styling	1 / 15	160 / 1
51	jade-browserify	H	General Programming	1 / 15	170 / 1
52	flashcanvas	H	General Programming	1 / 15	177 / 5

Continued on next page

Table 12 – Continued

No.	Package Name	Type	Task Domain	Models /Days	Downloads / IPs
53	beautifulsoup4	C	General Programming	1 / 15	350 / 95
54	apache-httpclient	C	General Programming	1 / 15	158 / 1
55	directx	C	Game Development	1 / 28	140 / 1
56	slimdx	C	Game Development	1 / 15	173 / 1
57	opentk	C	Game Development	1 / 15	168 / 1
58	windows-api-codec-pack	H	Web Development	0 / 15	168 / 5
59	sharpgl	C	Game Development	1 / 15	163 / 1
60	commons-net	C	General Programming	1 / 15	169 / 1
61	vue-ssr-devtools	H	UI & Styling	1 / 15	153 / 1
62	platformio	C	System & Hardware	1 / 15	197 / 1
63	arduinounit	C	UI & Styling	1 / 15	190 / 1
64	vim-refactor	H	General Programming	1 / 15	177 / 1
65	dayjs-plugin-duration	H	Configuration & Setup	1 / 15	728 / 1
66	node-jwt-simple	H	Security & Authentication	1 / 15	171 / 1
67	next-refresh-token	H	Security & Authentication	1 / 13	557 / 1
68	openssl-node	H	Security & Authentication	1 / 14	69 / 1
69	serve-static-corell	H	Web Development	1 / 13	67 / 1
70	vite-plugin-esm-import-extension	H	Configuration & Setup	1 / 12	69 / 1
71	nsubstitute	C	General Programming	6 / 12	66 / 1
72	hibernate-validator	C	Error & Troubleshooting	3 / 12	122 / 1
73	expo-firestore-upload	H	Database & Storage	1 / 12	73 / 1
74	expo-blob-util	H	Mobile Development	1 / 12	66 / 1
75	vite-plugin-legacy-umd	H	Configuration & Setup	1 / 12	71 / 1
76	vnc-viewer	C	System & Hardware	1 / 12	76 / 1
77	iokit	C	System & Hardware	4 / 12	66 / 1
78	core-foundation	C	System & Hardware	1 / 12	71 / 1
79	apache-iotdb	C	Database & Storage	5 / 12	0 / 0
80	ml-preprocessing	G	Data Processing & Analysis	1 / 12	0 / 0

Continued on next page

Table 12 – Continued

No.	Package Name	Type	Task Domain	Models /Days	Downloads / IPs
81	listenv	C	Data Processing & Analysis	1 / 12	0 / 0
82	dplyr	C	Data Processing & Analysis	7 / 12	30 / 1
83	lubridate	C	Data Processing & Analysis	7 / 12	0 / 0
84	uvicorn	C	Web Development	7 / 12	0 / 0
85	google-auth-httplib2	C	Security & Authentication	5 / 12	0 / 0
86	scikeras	C	General Programming	8 / 14	0 / 0
87	pyserial	C	System & Hardware	1 / 14	7 / 1
88	espresso-idling-resource	C	Testing & Quality	1 / 14	0 / 0
89	undetected-chromedriver	C	Testing & Quality	5 / 14	0 / 0
90	jasmine-marbers	H	Testing & Quality	1 / 14	0 / 0
91	apple-appstore-server-library	H	Mobile Development	1 / 14	0 / 0
92	pyiceberg	C	Database & Storage	5 / 14	16 / 1
93	iceberg-api	C	Web Development	1 / 14	0 / 0
94	pydantic	C	General Programming	7 / 14	0 / 0
95	sails-mongo-ng	H	Database & Storage	1 / 14	22 / 1
96	ohhttpstubs	C	Web Development	1 / 14	0 / 0
97	chromecast-caf-receiver	H	Testing & Quality	1 / 14	0 / 0
98	gunicorn	C	Web Development	4 / 14	0 / 0
99	boost-iterator	C	Performance & Optimization	1 / 14	0 / 0
100	boost-static-assert	C	Performance & Optimization	1 / 16	0 / 0
101	scrapy-user-agents	C	General Programming	1 / 14	0 / 0
102	karma-spec-filter	H	Testing & Quality	1 / 14	0 / 0
103	svelte-local-storage	H	Database & Storage	1 / 14	0 / 0
104	office-addin-debugger	H	Error & Troubleshooting	1 / 14	0 / 0
105	throttling-proxy	G	Performance & Optimization	1 / 28	0 / 0
106	rlang	C	Data Processing & Analysis	3 / 28	0 / 0
107	csvtool	C	Data Processing & Analysis	1 / 14	0 / 0

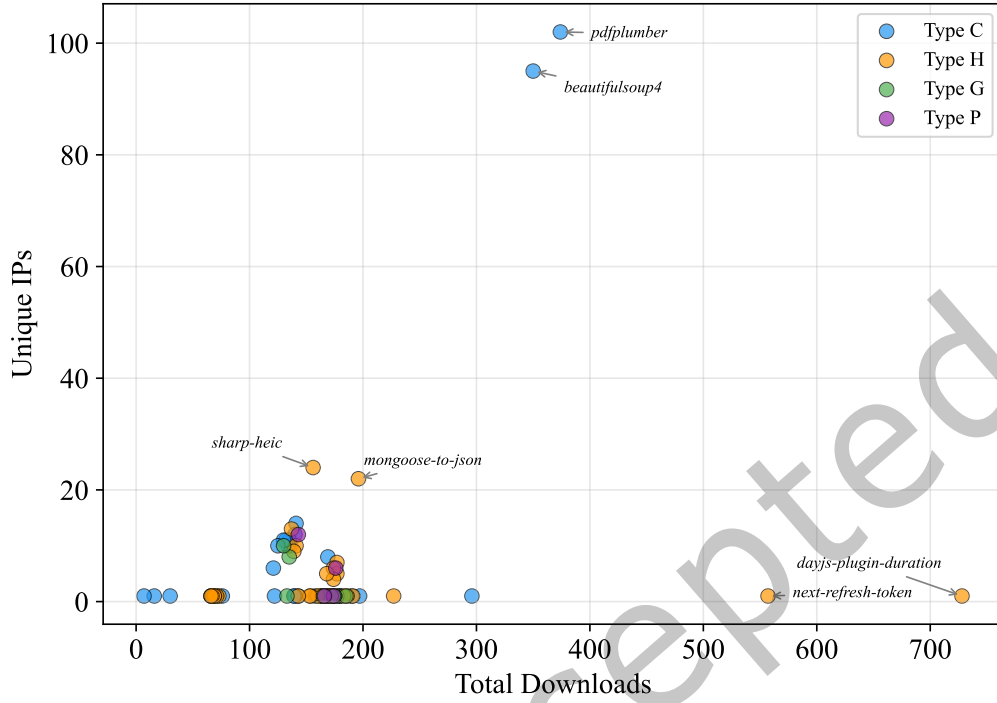


Fig. 10. Downloads and Unique IPs for Hallucinated Packages.

D Per-Model Hallucination Frequency Analysis

This appendix provides the detailed per-model generation frequency data for 107 hallucinated packages, as discussed in Section 5.5. Each of 107 original hallucinated packages was re-tested across 8 LLMs with 10 independent repetitions per (package, model) pair. The frequency for each pair is defined as the number of times, out of 10 trials, that the model reproduced hallucinated packages. Table 13 summarizes the overall hallucination frequency by model. Grok-3 exhibits the highest reproduction rate for 22.90%, while DeepSeek-R1 is the most robust for 2.90%.

Table 14 presents the per-model frequency matrix for the 20 highest-frequency hallucinated packages. Values represent the proportion of 10 trials in which the package was reproduced (e.g., 1.0 = 10/10 trials). We highlight three key observations. First, 61.7%, or 66 out of 107 hallucinated packages, were never reproduced, confirming their ephemeral nature. Second, only one package, namely scikeras, was reproduced by all eight models. Finally, Grok-3 dominates the high-frequency cells, achieving a perfect 10 out of 10 trials for 12 of the top 20 packages.

E Detailed McNemar Test Calculations for RQ5

This appendix provides the full derivation of the McNemar paired tests reported in Section 5.5. We matched all 91 prompts with the 8 models, which gave us a total of 728 pairs of prompts and models. For every pair under a given condition, we aggregate 10 repeated trials via majority vote: a pair is classified as *persisting* if hallucinated packages appear in ≥ 6 of the 10 trials. For two conditions X and Y , each pair falls into exactly one cell of a 2×2 contingency table $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$, where a : both persist, b : only X persists, c : only Y persists, d : neither persists. The

Table 13. Model-Level Hallucination Frequency Summary (10 Repetitions $\times$ 107 Packages per Model)

Model	Total Hits / 1070	Frequency (%)	Packages Hit
Grok-3-Preview	245	22.90	33
ChatGPT-4o	135	12.62	24
DeepSeek-V3	126	11.78	26
Gemini-2.0-Flash	94	8.79	19
Qwen3-235B	89	8.32	17
Gemini-2.5-Flash	83	7.76	18
GPT-4.1	46	4.30	9
DeepSeek-R1	31	2.90	14

Table 14. Per-Model Reproduction Frequency for Top-20 Hallucinated Packages

Package	GPT-4o	GPT-4.1	DSV3	DSR1	G2.5F	G2.0F	Grok3	Qw3	Avg	Models
scikeras	0.8	0.3	1.0	0.5	0.8	0.7	1.0	1.0	0.76	8/8
mongoose-to-json	1.0	0.0	1.0	0.3	1.0	1.0	1.0	0.8	0.76	7/8
uvicorn	1.0	0.0	1.0	0.0	1.0	0.9	1.0	0.9	0.73	7/8
shaderc	0.0	0.0	1.0	0.0	1.0	1.0	1.0	1.0	0.66	7/8
fanotify	1.0	0.0	1.0	0.1	1.0	1.0	1.0	0.0	0.64	6/8
nsubstitute	0.0	0.0	1.0	0.0	0.8	1.0	1.0	1.0	0.60	6/8
pydantic	0.0	0.0	1.0	0.0	0.5	0.9	1.0	1.0	0.55	7/8
lubridate	0.3	0.1	0.3	0.1	0.3	0.3	1.0	0.7	0.51	7/8
scatterpie	0.2	0.1	0.1	0.1	0.2	0.2	1.0	0.5	0.43	7/8
google-auth-httplib2	0.8	0.0	0.0	0.3	0.0	0.3	0.8	0.9	0.39	5/8
apache-iotdb	0.4	0.0	1.0	0.3	0.3	0.0	1.0	0.0	0.38	5/8
iokit	0.0	0.0	0.7	0.0	0.4	0.8	1.0	0.0	0.36	4/8
jna	0.5	0.0	0.6	0.5	0.0	0.2	0.9	0.0	0.34	5/8
gunicorn	0.2	0.8	0.0	0.0	0.0	0.0	1.0	0.6	0.33	4/8
pyiceberg	0.0	0.0	1.0	0.0	0.3	0.3	1.0	0.0	0.33	5/8
healenium-web	0.0	0.0	0.9	0.0	0.0	0.3	0.9	0.0	0.26	3/8
fasteners	0.0	0.0	0.9	0.0	0.2	0.3	0.6	0.0	0.25	4/8
phpseclib	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	0.25	4/8
pdfplumber	0.0	0.0	0.8	0.0	0.0	0.0	0.8	0.2	0.23	5/8
jossion	0.6	0.0	0.0	0.0	0.0	0.4	0.7	0.0	0.21	3/8

McNemar test with Yates' correction uses only the pairs with different outcomes.

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}, \quad \chi^2 \sim \chi^2(1).$$

We apply the Bonferroni correction for three pairwise comparisons, resulting in an adjusted significance level of $\alpha_{\text{adj}} = 0.05/3 \approx 0.0167$. Table 15 shows the three contingency tables side by side.

(1) C0 (Vanilla) vs C1 (Prompt Enhancement). Discordant pairs: $b+c = 12+23 = 35$. $\chi^2 = (|12 - 23| - 1)^2 / 35 = 100/35 = 2.857$; $p = 0.091 > 0.0167$. **Not significant** after Bonferroni correction.

Table 15. McNemar Contingency Tables for Three Pairwise Comparisons ($n = 728$ Prompt–Model Pairs Each)

(a) C0 vs. C1				(b) C0 vs. C2				(c) C1 vs. C2			
	C1P	C1N	Σ		C2P	C2N	Σ		C2P	C2N	Σ
C0P	57	12	69	C0P	56	13	69	C1P	60	20	80
C0N	23	636	659	C0N	65	594	659	C1N	61	587	648
Σ	80	648	728	Σ	121	607	728	Σ	121	607	728

Note. P stands for Persists, and N stands for Does Not Persist. The cells showing a change are b , where the hallucination is fixed, and c , where a new hallucination appears.

(2) **C0 (Vanilla) vs C2 (Web Search)**. Discordant pairs: $b + c = 13 + 65 = 78$. $\chi^2 = (|13 - 65| - 1)^2 / 78 = 2601 / 78 = 33.346$; $p < 0.001 \ll 0.0167$. **Highly significant.**

(3) **C1 (Prompt Enhancement) vs C2 (Web Search)**. Discordant pairs: $b + c = 20 + 61 = 81$. $\chi^2 = (|20 - 61| - 1)^2 / 81 = 1600 / 81 = 19.753$; $p < 0.001 \ll 0.0167$. **Highly significant.**

The results confirm that the C2 web search method significantly increases hallucination persistence compared to both the basic C0 baseline and the C1 prompt enhancement. In contrast, the prompt enhancement alone does not cause a statistically significant change.

F Detailed Reconnaissance Cost Breakdown

This appendix provides a detailed and itemized breakdown of the API usage costs incurred during the reconnaissance phase, as discussed in Section 5.6. Table 16 details the token usage, pricing models and the exact cost for each of the eight commercial LLMs evaluated in our study.

Table 16. Reconnaissance Phase Cost Details (8 Commercial LLM Models, N=10,893 Queries/Model)

Model Name	Parameters	I/O Pricing (\$/M tokens)	Total Cost (\$)	Open Source
DeepSeek-V3	671B	0.28/0.42	7.47	✓
DeepSeek-R1	685B	0.55/2.19	25.06	✓
Qwen3-235B-A22B	235B	0.22/0.88	10.07	✓
Grok-3-P	–	3/15	160.12	×
Gemini-2.0F	–	0.1/0.4	4.58	×
ChatGPT-4o-L	–	5/15	190.63	×
GPT-4.1-2025	–	2/8	91.49	×
Gemini-2.5F-P	–	0.15/0.6	6.86	×
Total		496.28		

Note: Assumes 1,400 input tokens and 700 output tokens per query (based on code generation tasks). Abbreviations: F = Flash, P = Preview, L = Latest.

Received 1 November 2025; revised 20 May 2026; accepted 30 June 2026